

گزارش پروژه تبدیل زبان Hash به Promela

فاطمه باقریان – امیرحسین پیرنژاد

تیر ماه ۱۴۰۵

فهرست مطالب

فاز اول	
۱.۰ مقدمه	۴
۲.۰ ارتباط با نظریه اتوماتا و زبان‌ها	۴
۳.۰ زیرمجموعه زبان Hash پشتیبانی شده	۴
۴.۰ محدودیت‌های پیاده‌سازی	۴
۵.۰ نحوه عملکرد کل سیستم	۵
۱ توضیح کلاس Main	۶
۱.۱ خواندن ورودی	۶
۲.۱ تولید Parser و Lexer	۶
۳.۱ بررسی خطاهای نحوی	۶
۴.۱ بررسی معنایی	۷
۵.۱ تولید کد Promela	۷
۶.۱ ذخیره خروجی	۷
۲ چرا از Visitor استفاده شده است؟	۸
۱.۲ مقایسه Visitor و Listener	۸
۲.۲ دلایل اصلی انتخاب Visitor	۸
۳.۲ نمونه‌های عملی از مزایای Visitor	۹
۳ طراحی (HashToPromela) Translator	۱۰
۱.۳ ساختار کلی کلاس	۱۰
۲.۳ متد visitProgram - نقطه ورود اصلی	۱۰
۳.۳ متد visitVarDecl - ترجمه تعریف متغیرها	۱۱
۴.۳ ترجمه عبارات (Expressions) در HashToPromela	۱۱
۱.۴.۳ روش کلی ترجمه	۱۱
۲.۴.۳ سلسله‌مراتب ترجمه عبارات	۱۲
۳.۴.۳ نحوه اتصال متدها	۱۲
۴.۴.۳ عملگرهای پشتیبانی‌شده	۱۲
۵.۴.۳ مدیریت عملگرهای خاص	۱۲
۶.۴.۳ مزایای روش Visitor	۱۳
۵.۳ متد visitAssignment - ترجمه انتساب	۱۳
۶.۳ متد visitIfStmt - ترجمه دستور شرطی	۱۴
۷.۳ متد visitWhileStmt - ترجمه حلقه while	۱۵
۸.۳ متد visitForStmt - ترجمه حلقه for	۱۷
۹.۳ متد visitTryStmt - ترجمه ساختار try-catch	۱۹

۲۰	توابع کمکی مهم	۴
۲۰	mapType - نگاشت انواع داده	۱.۴
۲۰	guardDivisionByZero - محافظت از تقسیم بر صفر	۲.۴
۲۲	findZeroDivisors - یافتن مقسوم‌علیه‌های صفر	۳.۴
۲۲	expandPower - تبدیل توان	۴.۴
۲۲	indent - فرمت‌بندی خروجی	۵.۴
۲۲	exitLoopFlags / enterLoopFlags - مدیریت پرچم‌های حلقه	۶.۴

۵ نتیجه‌گیری و جمع‌بندی ۲۳

۲۴	فاز دوم	
۲۴	مقدمه فاز دوم	۱.۵
۲۴	برنامه‌های مورد بررسی	۲.۵
۲۴	۱.۲.۵ مدل پایه (output.pml)	
۲۴	۲.۲.۵ برنامه‌های تکمیلی (برای سناریوهای نقض)	
۲۶	۳.۵ خصوصیت اول - Safety	
۲۶	۱.۳.۵ فرمول LTL	
۲۶	۲.۳.۵ چرا فرمول به این شکل است؟	
۲۷	۳.۳.۵ سناریوی تایید - مدل پایه	
۲۷	۴.۳.۵ سناریوی نقض - برنامه (الف)	
۲۸	۴.۵ خصوصیت دوم - Liveness	
۲۸	۱.۴.۵ فرمول LTL	
۲۹	۲.۴.۵ چرا فرمول به این شکل است؟	
۲۹	۳.۴.۵ سناریوی تایید - مدل پایه	
۳۰	۴.۴.۵ سناریوی نقض (تلاش) - برنامه (ج-۱)	
۳۱	۵.۵ خصوصیت سوم - Reachability	
۳۱	۱.۵.۵ فرمول LTL	
۳۲	۲.۵.۵ چرا فرمول به این شکل است؟	
۳۲	۳.۵.۵ سناریوی تایید - مدل پایه (انتظار: نقض یافت شود)	
۳۳	۴.۵.۵ سناریوی مکمل - برنامه (ج-۲) (انتظار: نقض یافت نشود)	
۳۴	۶.۵ خصوصیت چهارم - Deadlock Freedom	
۳۴	۱.۶.۵ دستور اجرا	
۳۴	۲.۶.۵ خروجی ترمینال (مسیر اجرای کامل)	
۳۵	۳.۶.۵ تفسیر	
۳۵	۷.۵ خصوصیت پنجم - Invariant	
۳۵	۱.۷.۵ فرمول LTL	
۳۵	۲.۷.۵ چرا فرمول به این شکل است؟	
۳۵	۳.۷.۵ سناریوی تایید - مدل پایه	
۳۶	۴.۷.۵ سناریوی نقض - برنامه (ب)	

۳۸	فاز سوم	
۳۸	مقدمه فاز سوم	۸.۵
۳۸	اثبات ریاضی قضیه با استقرا روی اشتقاق	۹.۵
۳۸	۱.۹.۵ بیان دقیق قضیه	
۳۸	۲.۹.۵ چرا استقرا روی اشتقاق، نه روی عدد تکرار؟	
۳۹	۳.۹.۵ پایه‌ی استقرا - قانون While-F	
۳۹	۴.۹.۵ گام استقرا - قانون While-T	
۴۰	۵.۹.۵ جمع‌بندی اثبات	
۴۰	۱۰.۵ بازنگری مفاهیم کلیدی	

۴۰	Big-Step	۱.۱۰.۵
۴۰	تعریف انواع پایه در Lean	۱۱.۵
۴۱	ارزیابی عبارات و به‌روزرسانی State	۱۲.۵
۴۲	رابطه‌ی BigStep	۱۳.۵
۴۳	Invariant Loop – قضیه‌ی انتخابی	۱۴.۵
۴۳	صورت رسمی قضیه	۱.۱۴.۵
۴۳	چرا این قضیه؟	۲.۱۴.۵
۴۴	کد کامل اثبات و توضیح هر قدم	۱۵.۵
۴۵	صورت‌بندی قضیه و ورود مقدمات	۱.۱۵.۵
۴۵	چرا یک لم کمکی (aux) لازم است؟	۲.۱۵.۵
۴۶	اتصال نهایی	۳.۱۵.۵
۴۶	یک نکته‌ی فنی درباره‌ی کامپایل	۴.۱۵.۵
۴۶	مثال گام‌به‌گام (شفاف‌سازی رفتار استقرا)	۱۶.۵
۴۶	جمع‌بندی فاز سوم	۱۷.۵

فاز اول

۱.۰ مقدمه

۲.۰ ارتباط با نظریه اتوماتا و زبان‌ها

- یک برنامه در حال اجرا را می‌توان به عنوان یک State Machine در نظر گرفت - هر وضعیت حافظه یک State است و هر دستور، یک Transition.
- خواصی که روی برنامه بررسی می‌شوند، با استفاده از Linear Temporal Logic (LTL) بیان می‌شوند.
- هر فرمول LTL معادل یک Büchi Automaton است - یک ماشین حالت محدود که روی کلمات نامتناهی کار می‌کند.
- ابزار SPIN این اتوماتا را با مدل سیستم ترکیب کرده و تمام حالت‌های ممکن را جستجو می‌کند.
- پس آنچه در این پروژه انجام می‌دهیم، عملاً همان نظریه‌ای است که در کلاس خوانده‌ایم را به صورت عملی و قابل لمس پیاده‌سازی می‌کنیم.

۳.۰ زیرمجموعه زبان Hash پشتیبانی شده

زیرمجموعه مورد نظر شامل موارد زیر است:

- متغیرهای از نوع adad و boole
- دستور شرطی age/bood/vagarna
- حلقه‌های ta و baraye به همراه shekan و edame
- ساختار gereftar/emtehan به صورت پایه (یک بلوک catch بدون تودرتو)
- عملگرهای محاسباتی و مقایسه‌ای تعریف شده در پروژه اول

۴.۰ محدودیت‌های پیاده‌سازی

- دستور shekan تنها از حلقه‌ای که مستقیماً داخل آن است خارج می‌شود، نه از حلقه‌های بیرونی.
- دستور edame اجرای بدنه فعلی را متوقف کرده و به شرط حلقه بازمی‌گردد.

- برای gereftar/emtehan، تنها یک بلوک gereftar پشتیبانی می‌شود و تودرتوی بلوک‌های emtehan در این پروژه لازم نیست.
- کلاس‌ها و متدها جزو این زیرمجموعه نیستند.

۵.۰ نحوه عملکرد کل سیستم

فرآیند اجرای پروژه به صورت زیر است:



در این معماری، ابتدا کد ورودی توسط Lexer به توکن‌ها تبدیل می‌شود. سپس Parser بر اساس گرامر تعریف شده، درخت نحوی (Parse Tree) را تولید می‌کند. پس از اطمینان از عدم وجود خطای نحوی، Semantic Checker قواعد معنایی (مانند تعریف متغیرها قبل از استفاده) را بررسی می‌کند. در نهایت، Visitor درخت را پیمایش کرده و کد معادل Promela تولید می‌کند.

فصل ۱

توضیح کلاس Main

کلاس Main نقطه شروع اجرای برنامه است و وظیفه هماهنگی بین بخش‌های مختلف را بر عهده دارد.

۱.۱ خواندن ورودی

```
CharStream input = CharStreams.fromPath(Paths.get(args[0]));
```

این خط فایل ورودی که به عنوان آرگومان خط فرمان دریافت شده است را به جریان کاراکتری تبدیل می‌کند. این جریان سپس توسط *Lexer* پردازش می‌شود. استفاده از *Paths.get* و *CharStreams.fromPath* روش استاندارد و امن برای خواندن فایل در *JAVA* است.

۲.۱ تولید *Lexer* و *Parser*

```
HashLexer lexer = new HashLexer(input);  
CommonTokenStream tokens = new CommonTokenStream(lexer);  
HashParser parser = new HashParser(tokens);
```

- *Lexer*: جریان کاراکتری را به توکن‌های معنادار (مانند کلیدواژه‌ها، عملگرها، شناسه‌ها و اعداد) تبدیل می‌کند.
- *TokenStream*: جریان توکن‌ها را به صورتی که *Parser* بتواند از آن استفاده کند، سازماندهی می‌کند.
- *Parser*: بر اساس گرامر تعریف شده در فایل *Hash.g4*، توکن‌ها را تحلیل کرده و درخت نحوی انتزاعی تولید می‌کند.

۳.۱ بررسی خطاهای نحوی

```
if (parser.getNumberOfSyntaxErrors() > 0) {  
    System.out.println("Syntax errors found. Semantic  
        checking skipped.");  
    return;  
}
```

در صورت وجود خطای نحوی، ادامه پردازش متوقف می‌شود. این کار از تولید خروجی ناقص یا اشتباه جلوگیری می‌کند. خطاهای نحوی ممکن است شامل استفاده از کلیدواژه‌های اشتباه، قرارگیری نادرست پرانتزها، یا ساختارهای غیرمجاز باشند.

۴.۱ بررسی معنایی

```
۱ SemanticChecker checker = new SemanticChecker();  
۲ checker.collectDef(tree);  
۳ checker.SemanticCheck(tree);
```

در این مرحله، دو عملیات مهم انجام می‌شود:

۱. **جمع‌آوری تعاریف:** collectDef تمام تعاریف متغیرها، توابع و کلاس‌ها را از درخت استخراج کرده و در یک جدول نماد (Symbol Table) ذخیره می‌کند.

۲. **بررسی معنایی:** SemanticCheck قواعد زیر را بررسی می‌کند:

- استفاده از متغیرها قبل از تعریف
- تطابق نوع داده در عملیات‌ها
- تعریف تکراری متغیرها
- استفاده صحیح از ساختارهای شرطی و حلقه‌ها
- و

اگر خطای معنایی وجود داشته باشد، پیام‌های خطا نمایش داده می‌شوند و برنامه ادامه نمی‌یابد.

۵.۱ تولید کد Promela

```
۱ HashToPromela translator = new HashToPromela();  
۲ String outputCode = translator.visit(tree);
```

در این مرحله، درخت نحوی توسط Visitor پیمایش می‌شود و برای هر گره، کد معادل Promela تولید می‌گردد. استفاده از Visitor به ما امکان می‌دهد تا دقیقاً کنترل کنیم که برای هر نوع گره چه کدی تولید شود.

۶.۱ ذخیره خروجی

```
۱ Path outputDir = Paths.get("src/codes");  
۲ Path outputFile = outputDir.resolve("output.pml");  
۳ Files.writeString(outputFile, outputCode);
```

کد تولید شده در فایل output.pml در پوشه src/codes ذخیره می‌شود. این مسیر استاندارد باعث می‌شود که خروجی به راحتی قابل دسترسی و استفاده در فازهای بعدی باشد.

فصل ۲

چرا از Visitor استفاده شده است؟

۱.۲ مقایسه Visitor و Listener

در این پروژه از الگوی Visitor به جای Listener استفاده شده است. جدول زیر مقایسه جامعی بین این دو الگو ارائه می‌دهد:

ویژگی	Visitor	Listener
مقدار بازگشتی	دارد (می‌تواند هر نوع داده‌ای برگرداند)	ندارد (void)
کنترل پیمایش	کامل (خودتان تصمیم می‌گیرید چه گره‌هایی را ببینید)	محدود (پیمایش خودکار توسط ANTLR)
کاربرد اصلی	تولید کد، تبدیل، تحلیل عمیق	تحلیل ساده، جمع‌آوری اطلاعات
وابستگی به گرامر	کم (هر گره یک متد مجزا دارد)	زیاد (همه در یک متد)
پیچیدگی پیاده‌سازی	بیشتر (نیاز به مدیریت پیمایش)	کمتر (پیمایش خودکار)
قابلیت توسعه	بالا (افزودن عملیات جدید بدون تغییر گره‌ها)	متوسط

جدول ۱.۲: مقایسه الگوهای Visitor و Listener

۲.۲ دلایل اصلی انتخاب Visitor

۱. **تولید مقدار بازگشتی:** مهم‌ترین دلیل، نیاز به تولید کد Promela به عنوان خروجی است. در Visitor، هر متد می‌تواند یک رشته (یا هر نوع داده دیگری) برگرداند، در حالی که Listener خروجی ندارد.
۲. **کنترل کامل بر پیمایش:** در Visitor، خودمان تصمیم می‌گیریم که چه گره‌هایی را بازدید کنیم و به چه ترتیبی. این ویژگی برای ترجمه دقیق دستورات Hash به Promela ضروری است، زیرا ترتیب اجرای دستورات و ساختارهای کنترلی باید به درستی حفظ شود.
۳. **بهبود قابلیت نگهداری:** با استفاده از Visitor، هر قانون گرامر یک متد مجزا دارد که کار اصلاح و توسعه را بسیار ساده‌تر می‌کند. اگر بخواهیم یک دستور جدید اضافه کنیم، فقط یک متد جدید می‌نویسیم.
۴. **مدیریت بهتر Context:** در فرآیند ترجمه، نیاز به نگهداری وضعیت‌هایی مانند برچسب‌های حلقه‌ها یا پرچم‌های استثنا داریم. Visitor با استفاده از متغیرهای عضو کلاس یا پشته‌ها، این کار را به سادگی امکان‌پذیر می‌کند.

۵. انتخابی بازدید از گره‌ها: در Visitor، می‌توانیم از بازدید برخی گره‌ها صرف‌نظر کنیم یا گره‌های خاصی را چندین بار بازدید کنیم، در حالی که در Listener همه گره‌ها دقیقاً یک بار بازدید می‌شوند.

۳.۲ نمونه‌های عملی از مزایای Visitor

- مدیریت تقسیم بر صفر: در متدهای مختلف (مانند visitWhileStmt)، می‌توانیم تصمیم بگیریم که قبل از شرط اصلی حلقه، یک شرط اضافی برای بررسی تقسیم بر صفر اضافه کنیم.
- مدیریت حلقه‌های تو در تو: با استفاده از پشته continueLabels، می‌توانیم به راحتی برچسب‌های حلقه‌های تو در تو را مدیریت کنیم. هر بار که وارد یک حلقه جدید می‌شویم، برچسب آن را به پشته اضافه می‌کنیم و پس از خروج، آن را حذف می‌کنیم.
- مدیریت استثناها: در ساختار try-catch، با استفاده از پرچم‌های استثنا که روی پشته قرار می‌گیرند، می‌توانیم رفتار استثنا را در Promela شبیه‌سازی کنیم.

فصل ۳

طراحی (HashToPromela) Translator

کلاس HashToPromela مسئول اصلی تبدیل کد Hash به Promela است. این کلاس از `HashBaseVisitor<String>` ارث‌بری کرده و برای هر گره از درخت نحوی، یک رشته خروجی تولید می‌کند.

۱.۳ ساختار کلی کلاس

• متغیرهای سراسری اضافه شده:

- `divByZero`: برای بررسی ویژگی Safety (جلوگیری از تقسیم بر صفر)
- `inLoop` و `exitLoop`: برای بررسی ویژگی‌های Liveness
- `endReached`: برای بررسی ویژگی Reachability
- `activeLoopCount`: برای مدیریت حلقه‌های تو در تو

• پشته‌ها:

- `continueLabels`: ذخیره برچسب‌های حلقه‌ها برای دستور `edame`
- `exceptionFlags`: ذخیره پرچم‌های استثنا برای ساختار `try-catch`
- `catchLabels`: ذخیره برچسب‌های بلوک‌های `catch`

• شمارنده‌ها:

- `loopCounter`: تولید برچسب‌های یکتا برای حلقه‌ها
- `exceptionCounter`: تولید شناسه‌های یکتا برای استثناها

۲.۳ متد `visitProgram` - نقطه ورود اصلی

این متد مهم‌ترین متد کلاس است و ساختار کلی فایل خروجی را تولید می‌کند:

۱. تعریف متغیرهای کمکی: در ابتدای خروجی، تمام متغیرهای کمکی که برای بررسی خواص سیستم نیاز هستند، تعریف می‌شوند:

```

1      bool divByZero = false;
2      bool inLoop = false;
3      bool exitLoop = false;
4      bool endReached = false;
5      int activeLoopCount = 0;

```

۲. پردازش متغیرهای سراسری: اعلان‌های سطح بالا که از نوع تعریف متغیر هستند، شناسایی شده و به صورت متغیرهای سراسری در Promela تعریف می‌شوند. این کار باعث می‌شود که متغیرها در کل برنامه قابل دسترسی باشند.

۳. تعریف بدنه اصلی: سایر گره‌های سطح بالا (دستورات، توابع، کلاس‌ها) درون یک پروسه فعال با نام main قرار می‌گیرند، همچنین متغیرهای سراسری در این قسمت مقدار دهی میشوند:

```

1      active proctype main() {
2          \\ body
3      }

```

۴. افزودن برچسب پایان: در انتهای بدنه، برچسب endReachedLabel و دستور تنظیم endReached = true اضافه می‌شود تا بتوان ویژگی Reachability را بررسی کرد.

۳.۳ متد visitVarDecl - ترجمه تعریف متغیرها

این متد وظیفه تبدیل اعلان متغیرها را بر عهده دارد:

۱. نگاشت نوع: نوع داده Hash به نوع معادل در Promela تبدیل می‌شود:

- $\text{int} \rightarrow \text{adad}$
- $\text{bool} \rightarrow \text{boole}$
- سایر انواع (مانند matn, harf) پشتیبانی نمی‌شوند

۲. مدیریت عملگرهای افزایش/کاهش: اگر عبارت اولیه شامل ++x یا x++ باشد، به صورت زیر ترجمه می‌شود (به طور مشابه برای --):

- برای ++x: ابتدا مقداردهی با مقدار فعلی، سپس افزایش
- برای x++: ابتدا افزایش، سپس مقداردهی با مقدار جدید

۳. بررسی تقسیم بر صفر: اگر عبارت شامل عملگر تقسیم باشد، تابع guardDivisionByZero فراخوانی می‌شود تا از تقسیم بر صفر جلوگیری کند.

۴.۳ ترجمه عبارات (Expressions) در HashToPromela

۱.۴.۳ روش کلی ترجمه

در کلاس HashToPromela، ترجمه عبارات با استفاده از الگوی Visitor و به صورت بازگشتی انجام می‌شود. به این معنی که هر عبارت بزرگ به زیرعبارت‌های کوچکتر تقسیم شده و هر کدام به صورت مستقل ترجمه می‌شوند، سپس نتایج با هم ترکیب می‌شوند.

۲.۴.۳ سلسله‌مراتب ترجمه عبارات

ترجمه عبارات از بالا به پایین و به ترتیب زیر انجام می‌شود:

visitExpr()	← Entry Point
visitLogicalOrExpr()	← Logical OR ()
visitLogicalAndExpr()	← Logical AND (&&)
visitEqualityExpr()	← Equality (==, !=)
visitRelationalExpr()	← Relational (<, >, <=, >=)
visitAdditiveExpr()	← Addition/Subtraction (+, -)
visitMultiplicativeExpr()	← Multiplication/Division (*, /, %)
visitPowerExpr()	← Power (**)
visitUnaryExpr()	← Unary (!, -, ++, --)
visitPostfixExpr()	← Postfix (++ , --)
visitPrimaryExpr()	← Primary (Numbers, Variables, Parentheses)

۳.۴.۳ نحوه اتصال متدها

هر متد Visitor یک رشته برمی‌گرداند و برای زیرعبارتها، متد visit مناسب را فراخوانی می‌کند. به عنوان مثال، متد visitAdditiveExpr برای ترجمه (a + b) به این صورت عمل می‌کند:

۱. ابتدا visit(ctx.multiplicativeExpr(0)) را صدا می‌زند که به "a" می‌رسد

۲. عملگر "+" را دریافت می‌کند

۳. سپس visit(ctx.multiplicativeExpr(1)) را صدا می‌زند که به "b" می‌رسد

۴. در نهایت "a + b" را برمی‌گرداند

۴.۴.۳ عملگرهای پشتیبانی‌شده

معادل Promela	عملگرهای Hash	دسته
+, -, *, / (ضرب متوالی) همان عملگرها ! , , && $x = x - 1$, $x = x + 1$	+, -, *, / , ** < , <= , > , >= & , , != $x --$, $x++$, $--x$, $++x$	حسابی مقایسه‌ای منطقی یکانی

جدول ۱.۳: عملگرهای پشتیبانی‌شده

۵.۴.۳ مدیریت عملگرهای خاص

• توان (**): با تابع expandPower به ضرب متوالی تبدیل می‌شود:

$$x * x * x \rightarrow x^{**3} -$$

• افزایش/کاهش (++ , -): ترتیب ارزیابی پیشوند و پسوند حفظ می‌شود:

$$- \rightarrow x++ \text{ ابتدا مقدار فعلی، سپس افزایش}$$

$$- \rightarrow ++x \text{ ابتدا افزایش، سپس مقدار جدید}$$

• تقسیم بر صفر: با تابع guardDivisionByZero مدیریت می‌شود

۶.۴.۳ مزایای روش Visitor

- کد تمیز و خواناتر: هر عملگر در متد جداگانه
- توسعه‌پذیری بالا: اضافه کردن عملگر جدید با اضافه کردن یک متد
- مدیریت خودکار بازگشت: ANTLR خودش تشخیص می‌دهد کدام متد صدا زده شود

۵.۳ متد visitAssignment - ترجمه انتساب

عملگرهای انتساب ساده و مرکب را پشتیبانی می‌کند:

```
1      switch (op) {
2          case "=": return left + " = " + right;
3          case "+=": return left + " = " + left + " + " + right;
4          case "-=": return left + " = " + left + " - " + right;
5          case "*=": return left + " = " + left + " * " + right;
6          case "/=": return left + " = " + left + " / " + right;
7      }
```

برای عملگر $/=$ ، بررسی تقسیم بر صفر اضافه می‌شود.
برای بررسی صحت ترجمه assignment و های expression حسابی، یک تست ساده با نام test01_basic.hash در نظر گرفته شد. هدف این تست آن است که نشان دهد تعریف متغیرهای عددی، مقداردهی اولیه، assignment و تقدم عملگرهای حسابی به درستی در خروجی Promela حفظ می‌شوند. در این مثال، سه متغیر عددی تعریف شده و سپس مقدار متغیر z با عبارت $x + y * 3$ محاسبه می‌شود. از آنجا که ضرب نسبت به جمع تقدم دارد، انتظار داریم همین ساختار در خروجی حفظ شود.
کد ورودی Hash برای این تست به صورت زیر است:

```
1      adad x = 1;
2      adad y = 2;
3      adad z = 0;
4
5      z = x + y * 3;
6
7      bechap(z);
8
```

خروجی تولیدشده توسط مترجم در فایل output.pml به صورت زیر است:

```
bool divByZero = false;
bool inLoop = false;
bool exitLoop = false;
bool endReached = false;
int activeLoopCount = 0;

int x;
int y;
int z;

active proctype main() {
    x = 1;
    y = 2;
```

```

    z = 0;
    z = x + y * 3;
    printf("%d\n", z);
endReachedLabel:
    endReached = true;
    skip;
}

```

در این خروجی، نوع `adad` به نوع `int` در `Promela` تبدیل شده است. متغیرهای `x`، `y` و `z` به صورت سراسری تعریف شده‌اند و مقداردهی اولیه آن‌ها داخل `active proctype main` انجام شده است. همچنین `assignment` مربوط به `z` به صورت `z = x + y * 3` تولید شده و معنای `expression` حفظ شده است. این خروجی نشان می‌دهد که مترجم در این تست، تعریف متغیر، مقداردهی اولیه، `assignment` و `expression` حسابی را به درستی ترجمه کرده است.

متغیرهای کمکی `divByZero`، `inLoop`، `exitLoop`، `endReached` و `activeLoopCount` نیز طبق طراحی کلی مترجم در ابتدای مدل اضافه شده‌اند. این متغیرها اگرچه در این تست ساده مستقیماً فعال نمی‌شوند، اما برای بررسی ویژگی‌های `Safety`، `Liveness` و `Reachability` در فاز دوم استفاده می‌شوند. در انتهای `main` نیز مقدار `endReached` برابر `true` می‌شود تا رسیدن اجرای برنامه به انتها در فاز دوم قابل بررسی باشد.

۶.۳ متد `visitIfStmt` - ترجمه دستور شرطی

دستور شرطی `age/bood/vagarna` به ساختار `fi ... -> ... :: else ... :: if` در `Promela` تبدیل می‌شود:

```

1  if
2  :: (condition) ->
3  // then block
4  :: else ->
5  // else block (optional)
6  fi

```

همچنین، اگر شرط شامل عملیات تقسیم باشد، یک شرط اضافی برای بررسی تقسیم بر صفر قبل از شرط اصلی اضافه می‌شود.

برای بررسی صحت ترجمه دستور شرطی، تستی با نام `test02_if_else.hash` در نظر گرفته شد. هدف این تست آن است که نشان دهد ساختار شرطی `age/bood/vagarna` در زبان `Hash` به درستی به ساختار شرطی `if/fi` در `Promela` تبدیل می‌شود. در این مثال، مقدار متغیر `x` برابر ۳ است و شرط `x > 0` بررسی می‌شود. اگر شرط برقرار باشد، مقدار `y` برابر ۱ می‌شود و در غیر این صورت مقدار آن برابر ۲ قرار می‌گیرد.

کد ورودی `Hash` برای این تست به صورت زیر است:

```

1  adad x = 3;
2  adad y = 0;
3
4  age (x > 0) bood {
5      y = 1;
6  } vagarna {
7      y = 2;
8  }
9
10 bechap(y);

```

خروجی تولیدشده توسط مترجم در فایل output.pml به صورت زیر است:

```
bool divByZero = false;
bool inLoop = false;
bool exitLoop = false;
bool endReached = false;
int activeLoopCount = 0;

int x;
int y;

active proctype main() {
    x = 3;
    y = 0;
    if
        :: (x > 0) ->
            y = 1;
        :: else ->
            y = 2;
    fi
    printf("%d\n", y);
endReachedLabel:
    endReached = true;
    skip;
}
```

در این خروجی، نوع adad برای متغیرهای x و y به نوع int در Promela تبدیل شده است. همچنین مقداردهی اولیه متغیرها داخل active proctype main انجام شده است. ساختار شرطی age/bood/vagarna نیز به ساختار command guarded در Promela تبدیل شده است؛ به این صورت که شرط $x > 0$ در شاخه اول قرار گرفته و در صورت برقرار بودن آن، دستور $y = 1$ اجرا می‌شود. شاخه else نیز معادل بخش vagarna در زبان Hash است و در صورت برقرار نبودن شرط، مقدار y را برابر ۲ قرار می‌دهد.

این تست نشان می‌دهد که مترجم می‌تواند ساختار شرطی ساده را بدون تغییر معنایی به Promela منتقل کند. همچنین وجود printf نشان می‌دهد که دستور bechap(y) به خروجی متنی مناسب در Promela تبدیل شده است. مانند سایر خروجی‌ها، متغیرهای کمکی divByZero، inLoop، exitLoop، activeLoopCount و endReached نیز در ابتدای مدل اضافه شده‌اند تا در فاز دوم برای بررسی ویژگی‌های مختلف استفاده شوند. در انتهای main نیز مقدار endReached برابر true می‌شود تا رسیدن برنامه به انتها قابل بررسی باشد.

۷.۳ متد visitWhileStmt - ترجمه حلقه while

حلقه ta به ساختار `break od -> else :: ... :: do` تبدیل می‌شود:

۱. ایجاد برچسب: یک برچسب یکتا برای شروع حلقه تولید می‌شود (مثلاً loopStartNo_1).

۲. افزایش شمارنده: activeLoopCount یک واحد افزایش می‌یابد.

۳. ترجمه شرط: شرط حلقه به یک شرط در ساختار do تبدیل می‌شود.

۴. مدیریت پرچم‌ها: قبل از بدنه حلقه، پرچم‌های inLoop و exitLoop تنظیم می‌شوند.

۵. بررسی تقسیم بر صفر: یک شرط اضافی برای تشخیص تقسیم بر صفر قبل از شرط اصلی اضافه می‌شود.

۶. پس از خروج: پس از خروج از حلقه، activeLoopCount کاهش یافته و پرچم‌ها به‌روز می‌شوند.

برای بررسی صحت ترجمه حلقه ta و همچنین دستورات shekan و edame، تست سوم با نام hash.test03_while_break_continue طراحی شد. هدف این تست آن است که نشان دهد حلقه ta به ساختار do/od در Promela تبدیل می‌شود و دستورات edame و shekan نیز به ترتیب با goto و break مدل می‌شوند.

نام فایل این تست به صورت زیر است:

در این برنامه، متغیر i در هر تکرار یک واحد افزایش می‌یابد. اگر مقدار i برابر ۲ شود، دستور edame اجرا می‌شود و اجرای باقی بدنه همان تکرار متوقف شده و کنترل به ابتدای حلقه بازمی‌گردد. همچنین اگر مقدار i برابر ۴ شود، دستور shekan اجرا شده و حلقه خاتمه می‌یابد. کد ورودی Hash برای این تست به صورت زیر است:

```
1      adad i = 0;
2      adad s = 0;
3
4      ta (i < 5) {
5          i = i + 1;
6
7          age (i == 2) bood {
8              edame;
9          } vagarna {
10             s = s + i;
11         }
12
13         age (i == 4) bood {
14             shekan;
15         } vagarna {
16             s = s;
17         }
18     }
19
20     bechap(s);
21
```

خروجی تولیدشده توسط مترجم در فایل output.pml به صورت زیر است:

```
bool divByZero = false;
bool inLoop = false;
bool exitLoop = false;
bool endReached = false;
int activeLoopCount = 0;

int i;
int s;

active proctype main() {
    i = 0;
    s = 0;
```

```

        activeLoopCount = activeLoopCount + 1;
        loopStartNo_1:
        do
            :: (i < 5) ->
            inLoop = true;
            exitLoop = false;
            i = i + 1;
            if
                :: (i == 2) ->
                goto loopStartNo_1;
            :: else ->
            s = s + i;
            fi
            if
                :: (i == 4) ->
                break;
            :: else ->
            s = s;
            fi
            :: else -> break
        od
        activeLoopCount = activeLoopCount - 1;
        exitLoop = true;
        if
            :: (activeLoopCount == 0) ->
            inLoop = false;
            :: else ->
            inLoop = true;
            fi
        printf("%d\n", s);
        endReachedLabel:
        endReached = true;
        skip;
    }

```

۸.۳ متد visitForStmt - ترجمه حلقه for

حلقه baraye به ساختار زیر در Promela تبدیل می‌شود:

```

۱ // Initialization
۲ activeLoopCount = activeLoopCount + 1;
۳ do
۴ :: (condition) ->
۵ // Set flags
۶ // Loop body
۷ updateLabel:
۸ // Update
۹ :: else -> break
۱۰ od

```

```
// Reset flags after exit
```

برای بررسی صحت ترجمه حلقه baraye و رفتار دستور edame داخل حلقه for، تست چهارم با نام test04_for_continue.hash طراحی شد. هدف این تست آن است که نشان دهد حلقه baraye به ساختار شامل مقداردهی اولیه، شرط، بدنه، بخش به‌روزرسانی و خروج در Promela تبدیل می‌شود. همچنین این تست بررسی می‌کند که دستور edame در حلقه baraye برخلاف حلقه ta نباید مستقیماً به ابتدای حلقه برود، بلکه باید ابتدا به بخش update منتقل شود. در این برنامه، متغیر x ابتدا برابر صفر قرار می‌گیرد. سپس یک حلقه baraye با متغیر j اجرا می‌شود. اگر مقدار j برابر ۱ شود، دستور edame اجرا می‌شود و در نتیجه دستور $x = x + j$ در آن تکرار اجرا نخواهد شد. اما قبل از بررسی دوباره شرط حلقه، بخش به‌روزرسانی یعنی $j = j + 1$ باید اجرا شود.

کد ورودی Hash برای این تست به صورت زیر است:

```
1      adad x = 0;
2
3      baraye (adad j = 0; j < 3; j = j + 1) {
4          age (j == 1) bood {
5              edame;
6          } vagarna {
7              x = x + j;
8          }
9      }
10
11      bechap(x);
12
```

خروجی تولیدشده توسط مترجم در فایل output.pml به صورت زیر است:

```
bool divByZero = false;
bool inLoop = false;
bool exitLoop = false;
bool endReached = false;
int activeLoopCount = 0;

int x;

active proctype main() {
    x = 0;
    int j;
    j = 0;
    activeLoopCount = activeLoopCount + 1;
    do
    :: (j < 3) ->
        inLoop = true;
        exitLoop = false;
        if
        :: (j == 1) ->
            goto loopUpdate_1;
        :: else ->
            x = x + j;
        fi
    loopUpdate_1:
```

```

j = j + 1;
:: else -> break
od
activeLoopCount = activeLoopCount - 1;
exitLoop = true;
if
:: (activeLoopCount == 0) ->
inLoop = false;
:: else ->
inLoop = true;
fi
printf("%d\n", x);
endReachedLabel:
endReached = true;
skip;
}

```

در این خروجی، متغیر x به صورت سراسری تعریف شده و مقدار اولیه آن داخل active proctype قرار گرفته است. متغیر z که مربوط به حلقه baraye است، داخل main تعریف و سپس با مقدار صفر مقداردهی شده است. بعد از مقداردهی اولیه، مقدار activeLoopCount یک واحد افزایش می‌یابد تا ورود به یک حلقه فعال ثبت شود.

ساختار حلقه baraye به صورت یک do/od در Promela تولید شده است. شرط $z < 3$ در شاخه اصلی حلقه قرار گرفته و در صورتی که این شرط برقرار باشد، بدنه حلقه اجرا می‌شود. اگر شرط برقرار نباشد، شاخه else -> break فعال شده و اجرای حلقه پایان می‌یابد.

نکته اصلی این تست در ترجمه دستور edame دیده می‌شود. در خروجی، دستور edame به صورت goto loopUpdate_1 ترجمه شده است. این یعنی اگر شرط $z == 1$ برقرار شود، برنامه مستقیماً به برچسب loopUpdate_1 می‌رود و ابتدا دستور $z = z + 1$ اجرا می‌شود. سپس اجرای حلقه ادامه پیدا می‌کند و شرط حلقه دوباره بررسی می‌شود. این رفتار دقیقاً معادل رفتار continue در حلقه for است. پس از خروج از حلقه، مقدار activeLoopCount کاهش می‌یابد و مقدار exitLoop برابر true قرار می‌گیرد. سپس بررسی می‌شود که آیا هنوز حلقه فعال دیگری وجود دارد یا نه. اگر مقدار activeLoopCount برابر صفر باشد، مقدار inLoop برابر false می‌شود؛ در غیر این صورت، inLoop برابر true باقی می‌ماند. این بخش برای پشتیبانی از حلقه‌های تودرتو و بررسی ویژگی Liveness در فاز دوم استفاده می‌شود.

این تست نشان می‌دهد که مترجم می‌تواند حلقه baraye، مقداردهی اولیه، شرط، بدنه، بخش به‌روزرسانی و رفتار صحیح edame در حلقه for را به درستی به مدل Promela تبدیل کند.

۹.۳ متد visitTryStmt - ترجمه ساختار try-catch

از آنجا که Promela از مفهوم استثنا پشتیبانی نمی‌کند، این ساختار با استفاده از پرچم‌ها و برچسب‌ها شبیه‌سازی می‌شود:

۱. **تعریف پرچم:** یک پرچم جدید مانند errFlag_1 با مقدار false تعریف می‌شود.
۲. **اجرای بلوک try:** بدنه emtehan اجرا می‌شود. در صورت بروز خطا (تقسیم بر صفر)، پرچم به true تغییر کرده و به برچسب catchCheck_1 پرش می‌شود.
۳. **بررسی پرچم:** پس از بلوک try، پرچم بررسی می‌شود و اگر true باشد، بلوک catch اجرا می‌شود.

فصل ۴

توابع کمکی مهم

۱.۴ mapType - نگاشت انواع داده

تبدیل انواع داده Hash به Promela:

```
1 private String mapType(String hashType) {
2     switch (hashType) {
3         case "adad": return "int";
4         case "boole": return "bool";
5         default:     return "";
6     }
7 }
```

۲.۴ guardDivisionByZero - محافظت از تقسیم بر صفر

این تابع با دریافت لیست مقسوم‌علیه‌های مشکوک، یک ساختار شرطی تولید می‌کند:

```
1 if
2 :: (divisor1 == 0 || divisor2 == 0) ->
3   divByZero = true;
4   //      try-catch
5   errFlag = true;
6   goto catchLabel;
7 :: else ->
8   //
9   fi
```

برای بررسی صحت تولید گارد تقسیم بر صفر، تست پنجم با نام `test05_division_guard.hash` طراحی شد. هدف این تست آن است که نشان دهد اگر در برنامه Hash یک عملیات تقسیم وجود داشته باشد، مترجم قبل از اجرای تقسیم، مقدار مقسوم‌علیه را بررسی می‌کند. در این تست، مقدار متغیر `y` از ابتدا برابر صفر قرار داده شده است؛ بنابراین مسیر خطای تقسیم بر صفر باید در خروجی Promela قابل مشاهده باشد.

کد ورودی Hash برای این تست به صورت زیر است:

```
1 adad x = 10;
2 adad y = 0;
3 adad z = 0;
```

```

4
5      z = x / y;
6
7      bechap(z);
8

```

خروجی تولیدشده توسط مترجم در فایل output.pml به صورت زیر است:

```

bool divByZero = false;
bool inLoop = false;
bool exitLoop = false;
bool endReached = false;
int activeLoopCount = 0;

int x;
int y;
int z;

active proctype main() {
    x = 10;
    y = 0;
    z = 0;
    if
    :: ((y == 0)) ->
    divByZero = true;
    :: else ->
    z = x / y;
    fi
    printf("%d\n", z);
    endReachedLabel:
    endReached = true;
    skip;
}

```

در این خروجی، متغیرهای x ، y و z به صورت سراسری تعریف شده‌اند و مقداردهی اولیه آن‌ها داخل active proctype main انجام شده است. نکته اصلی این تست در ترجمه دستور $z = x / y$ دیده می‌شود. از آنجا که این دستور شامل عملگر تقسیم است، مترجم آن را مستقیم به $z = x / y$ تبدیل نکرده، بلکه ابتدا یک ساختار شرطی برای بررسی صفر بودن مقسوم‌علیه تولید کرده است. در شاخه اول، شرط $y == 0$ بررسی می‌شود. اگر این شرط برقرار باشد، مقدار divByZero برابر true قرار می‌گیرد. در این حالت دستور تقسیم اجرا نمی‌شود و بنابراین مدل به صورت مستقیم وارد عملیات نامعتبر تقسیم بر صفر نمی‌شود. در شاخه else، یعنی زمانی که y صفر نباشد، دستور اصلی $z = x / y$ اجرا می‌شود.

این تست نشان می‌دهد که تابع guardDivisionByZero به درستی برای هایی assignment که شامل تقسیم هستند اعمال می‌شود. همچنین چون این تقسیم خارج از ساختار emtehan/gereftar قرار دارد، در خروجی فقط متغیر divByZero فعال می‌شود و خبری از errFlag یا پرش به catchCheck نیست. این رفتار با طراحی مترجم سازگار است، زیرا errFlag فقط زمانی لازم است که تقسیم بر صفر داخل بلوک emtehan رخ دهد.

این خروجی برای فاز دوم نیز مهم است، چون ویژگی Safety با استفاده از متغیر divByZero بررسی می‌شود. در این تست، چون مقدار y برابر صفر است، انتظار می‌رود اگر فرمول noDivByZero روی این مدل اجرا شود، SPIN یک نقض پیدا کند.

۳.۴ findZeroDivisors - یافتن مقسوم‌علیه‌های صفر

این تابع با پیمایش بازگشتی درخت عبارت، تمام مقسوم‌علیه‌های عملیات تقسیم یا باقی‌مانده را جمع‌آوری می‌کند. به این صورت که در گره‌های MultiplicativeExprContext (شامل ضرب، تقسیم و باقی‌مانده)، عملگرها را بررسی کرده و اگر عملگر / یا % باشد، عبارت سمت راست را به عنوان یک مقسوم‌علیه مشکوک به لیست اضافه می‌کند. برای سایر گره‌ها، به صورت بازگشتی به فرزندان رفته و جستجو را ادامه می‌دهد. برای مثال، عبارت $(a / b / c) + (e / d)$ خروجی ["b", "c", "d"] را تولید می‌کند (چون در $a/b/c$ ، مقسوم‌علیه‌های b و c هر دو جمع‌آوری می‌شوند). این لیست سپس برای ساخت شرط بررسی صفر بودن مقسوم‌علیه‌ها استفاده می‌شود. این تابع در متدهای visitVarDecl، visitAssignmentStmt، visitIfStmt، visitWhileStmt و visitForStmt برای محافظت از تقسیم بر صفر به کار می‌رود.

۴.۴ expandPower - تبدیل توان

از آنجا که Promela از عملگر توان پشتیبانی نمی‌کند، $x^{**}n$ به $x*x*x*...$ (تکرار n بار) تبدیل می‌شود:

```
1 private String expandPower(String base, int exponent) {
2     StringBuilder sb = new StringBuilder("(");
3     for (int i = 0; i < exponent; i++) {
4         if (i > 0) sb.append(" * ");
5         sb.append("(").append(base).append(")");
6     }
7     sb.append(")");
8     return sb.toString();
9 }
```

۵.۴ indent - فرمت‌بندی خروجی

برای افزایش خوانایی کد خروجی، به هر خط ۴ فاصله اضافه می‌کند:

```
1 private String indent(String code) {
2     StringBuilder out = new StringBuilder();
3     for (String line : code.split("\n")) {
4         if (!line.isBlank()) {
5             out.append("    ").append(line);
6         }
7         out.append("\n");
8     }
9     return out.toString();
10 }
```

۶.۴ exitLoopFlags / enterLoopFlags - مدیریت پرچم‌های حلقه

این توابع کد مربوط به تنظیم و بازنشانی پرچم‌های inLoop و exitLoop را تولید می‌کنند که برای بررسی ویژگی‌های Liveness ضروری هستند.

فصل ۵

نتیجه‌گیری و جمع‌بندی

در این پروژه، یک ترنسلیتور کامل و کارآمد از زبان آموزشی Hash به زبان Promela طراحی و پیاده‌سازی شد. دستاوردهای اصلی این فاز عبارتند از:

۱. **ترجمه دقیق:** تمام ساختارهای زبانی مورد نیاز به درستی به Promela ترجمه می‌شوند.
۲. **مدیریت خواص سیستم:** با افزودن متغیرهای کمکی مانند `divByZero`، `inLoop` و `endReached`، زیرساخت لازم برای بررسی خواص مختلف در فازهای بعدی فراهم شده است.
۳. **پشتیبانی از استثناها:** با وجود اینکه Promela از استثنا پشتیبانی نمی‌کند، با استفاده از تکنیک‌های شبیه‌سازی، ساختار `try-catch` پشتیبانی می‌شود.
۴. **کد خوانا و قابل نگهداری:** استفاده از الگوی `Visitor` و توابع کمکی مناسب، کد را خوانا، ماژولار و به راحتی قابل توسعه کرده است.
۵. **پشتیبانی از عملگرهای پیشرفته:** عملگرهای توان، افزایش/کاهش، و انتساب مرکب به درستی ترجمه می‌شوند.

خروجی تولید شده در این فاز (فایل `output.pml`) به عنوان ورودی فاز دوم (Model Checking) استفاده خواهد شد. در فاز دوم، با استفاده از ابزار SPIN، پنج ویژگی مختلف روی این مدل بررسی می‌شوند تا صحت سیستم تأیید یا رد شود.

• **ویژگی Safety:** جلوگیری از تقسیم بر صفر (`divByZero`)

• **ویژگی Liveness:** خروج از حلقه‌ها (`exitLoop → inLoop`)

• **ویژگی Reachability:** رسیدن به پایان برنامه (`endReached`)

• **ویژگی Freedom Deadlock:** عدم وقوع بن‌بست

• **ویژگی Invariant:** همواره غیرمنفی بودن یک متغیر

این پروژه نشان می‌دهد که چگونه مفاهیم تئوری زبان‌ها و ماشین‌ها (اتوماتا، گرامرها، و معناشناسی) در عمل و برای تأیید صحت نرم‌افزارهای واقعی به کار گرفته می‌شوند. توانایی اثبات ریاضی درستی نرم‌افزارها، یکی از مهارت‌های کلیدی در صنعت نرم‌افزار مدرن است.

فاز دوم

۱.۵ مقدمه فاز دوم

در فاز اول، برنامه‌ی زبان Hash به مدل Promela ترجمه شد (فایل output.pml). هدف این فاز، بررسی پنج خصوصیت مشخص – Invariant و Deadlock Freedom، Reachability، Liveness، Safety – بر روی این مدل با استفاده از ابزار SPIN است. برای هر خصوصیت، فرمول LTL متناظر (در صورت نیاز)، دستور دقیق اجرا، خروجی ترمینال و یک تفسیر مستقل ارائه شده است. تمامی خصوصیات با فرمول‌های named در فایل properties.ltl تعریف شده‌اند و با دستور `spin -search -ltl <name> verify.pml` اجرا شده‌اند.

روش‌شناسی بررسی. یکی از نکات مهمی که در طول این فاز به آن رسیدیم این است که تأیید یک خصوصیت روی یک برنامه‌ی ثابت، به تنهایی کافی نیست. اگر مسیر خطا یا مسیر بحرانی در آن برنامه‌ی خاص از اساس غیرقابل دسترس باشد، نتیجه‌ی 0 errors که SPIN برمی‌گرداند بدیهی (vacuously true) خواهد بود و چیزی درباره‌ی درستی واقعی مکانیزم تحت بررسی ثابت نمی‌کند. برای پرهیز از این مشکل، برای هر یک از چهار خصوصیت Safety، Liveness، Reachability و Invariant، دو سناریو طراحی و اجرا شده است:

- **سناریوی تایید:** روی مدل پایه (برنامه‌ی اصلی فاز اول)، که در آن انتظار می‌رود خصوصیت برقرار باشد.

- **سناریوی نقض:** روی یک برنامه‌ی طراحی‌شده که مسیر بحرانی در آن واقعاً reachable است، تا نشان داده شود SPIN می‌تواند نقض واقعی را هم تشخیص دهد (نه اینکه صرفاً همیشه pass بدهد).

خصوصیت Deadlock Freedom از این قاعده مستثناست؛ دلیل آن در بخش مربوطه توضیح داده شده است.

۲.۵ برنامه‌های مورد بررسی

۱.۲.۵ مدل پایه (output.pml)

مدل Promela اصلی که از ترجمه‌ی برنامه‌ی فاز اول به دست آمده، شامل یک عملیات تقسیم محافظت‌شده با سازوکار gereftar/emtehan (با $y=2$ ثابت)، دو حلقه‌ی متوالی و کراندار (اولی با شرط $i < 5$ ، دومی با شرط $j < 3$)، و یک برچسب پایانی endReachedLabel است. متغیرهای کنترلی inLoop، divByZero، exitLoop و endReached برای امکان بیان پنج خصوصیت در سطح سراسری (global) تعریف شده‌اند.

۲.۲.۵ برنامه‌های تکمیلی (برای سناریوهای نقض)

برای اینکه هر خصوصیت به صورت غیر بدیهی هم بررسی شود، سه برنامه‌ی Hash دیگر طراحی و به طور جداگانه به Promela ترجمه و با SPIN بررسی شدند:

برنامه (الف) – تقسیم بر صفر مستقیم (برای نقض Safety):

```

1      adad x = 10;
2      adad y = 0;
3      adad safe = 0;
4
5      emtehan {
6          safe = x / y;
7      }
8      gereftar (SefrBood e) {
9          safe = 0;
10     }
11
12     bechap(safe);
13

```

برنامه (ب) – کاهش پیوسته‌ی متغیر (برای نقض Invariant):

```

1      adad x = 3;
2      adad i = 0;
3
4      ta (i < 10) {
5          x = x - 1;
6          i = i + 1;
7      }
8
9      bechap(x);
10

```

برنامه (ج-۱) – حلقه‌ی نامتناهی با متغیر متغیر (برای نقض Liveness):

```

1      adad x = 10;
2      adad y = 2;
3      adad i = 0;
4      adad safe = 0;
5
6      ta (i < 5) {
7          x = x - 1;
8      }
9
10     baraye (adad j = 0; j < 3; j = j) {
11         safe = safe + j;
12     }
13
14     bechap(safe);
15

```

در این برنامه، شرط حلقه‌ی اول ($i < 5$) هیچ‌گاه به false نمی‌رسد چون i هرگز افزایش نمی‌یابد؛ بنابراین اجرای برنامه برای همیشه در حلقه‌ی اول باقی می‌ماند و به بدنه‌ی حلقه‌ی دوم یا خط bechap هرگز نمی‌رسد. نکته‌ی مهم اینجا است که در هر تکرار، x یک واحد کم می‌شود؛ یعنی علاوه بر نامتناهی بودن حلقه، مقدار متغیر x هم هیچ‌گاه تکرار نمی‌شود (به سمت منفی بی‌نهایت می‌رود).

برنامه (ج-۲) – حلقه‌ی نامتناهی با state ثابت (برای بررسی مکمل Reachability):

```

1      adad i = 0;
2      adad safe = 0;

```

```

3      adad x = 0;
4
5      ta (i < 5) {
6          safe = safe;
7      }
8
9      baraye (adad j = 0; j < 3; j = j) {
10         safe = safe + j;
11     }
12
13     bechap(safe);
14

```

این برنامه هم مثل (ج-۱) در حلقه‌ی اول برای همیشه باقی می‌ماند، با این تفاوت مهم که بدنه‌ی حلقه (safe = safe;) هیچ متغیری را واقعاً تغییر نمی‌دهد؛ در نتیجه state کلی برنامه پس از اولین تکرار دقیقاً تکرار می‌شود (self-loop).

۳.۵ خصوصیت اول – Safety

۱.۳.۵ فرمول LTL

$$\Box \neg(\text{divByZero})$$

معادل آن در properties.ltl: `ltl noDivByZero: [] (! (divByZero))`

۲.۳.۵ چرا فرمول به این شکل است؟

خصوصیت‌های Safety معمولاً به صورت $\Box \neg(\text{bad})$ بیان می‌شوند، زیرا هدف آن‌ها تضمین این است که سیستم در هیچ‌یک از حالت‌های قابل دسترس خود وارد وضعیت نامطلوب (bad state) نشود. در این پروژه، وقوع تقسیم بر صفر به‌عنوان وضعیت نامطلوب در نظر گرفته شده است؛ بنابراین از فرمول

$$\Box(\neg \text{divByZero})$$

استفاده شده است.

اگر به‌جای عملگر $\Box$ از $\Diamond$ استفاده می‌شد، یعنی

$$\Diamond(\neg \text{divByZero})$$

این فرمول تنها بیان می‌کرد که حداقل یک حالت در اجرای سیستم وجود دارد که در آن تقسیم بر صفر رخ نداده است. بدیهی است چنین شرطی حتی در صورتی که سیستم در ادامه اجرا وارد حالت خطا شود نیز برقرار خواهد بود؛ زیرا وجود تنها یک حالت بدون خطا برای صدق این فرمول کافی است. در مقابل، عملگر $\Box$ بیان می‌کند که شرط مورد نظر باید در تمام حالت‌های قابل دسترس برقرار باشد. از این‌رو، این عملگر انتخاب مناسبی برای بیان ویژگی‌های ایمنی است و تضمین می‌کند که در هیچ مرحله‌ای از اجرای سیستم تقسیم بر صفر رخ نخواهد داد.

۳.۳.۵ سناریوی تایید – مدل پایه

دستور اجرا

```
spin -search -ltl noDivByZero verify.pml
```

خروجی ترمینال

```
1      (Spin Version 6.5.1 -- 31 July 2020)
2      + Partial Order Reduction
3
4      Full statespace search for:
5      never claim          + (noDivByZero)
6      assertion violations + (if within scope of claim)
7      acceptance cycles   + (fairness disabled)
8      invalid end states  - (disabled by never claim)
9
10     State-vector 48 byte, depth reached 137, errors: 0
11     69 states, stored
12     1 states, matched
13     70 transitions (= stored+matched)
14     0 atomic steps
15     hash conflicts: 0 (resolved)
16
17     unreachable in proctype main
18     output.pml:20, state 7, "divByZero = 1"
19     output.pml:21, state 8, "errFlag_1 = 1"
20     output.pml:29, state 15, "safe = 0"
21     (6 of 69 states)
22     unreachable in claim noDivByZero
23     _spin_nvr.tmp:8, state 10, "-end-"
24     (1 of 10 states)
25
26     pan: elapsed time 0.003 seconds
27
```

تفسیر

جستجوی کامل فضای حالت هیچ نقضی برای فرمول $\neg(\text{divByZero})$ پیدا نکرد (errors: 0). با این حال، بخش unreachable نشان می‌دهد خط $\text{divByZero} = 1$ اصلاً کاوش نشده است؛ چون در این مدل $y=2$ ثابت است و هیچ مسیری برای رسیدن آن به صفر وجود ندارد. بنابراین این نتیجه به تنهایی بدیهی است و برای اطمینان از درستی واقعی گارد تقسیم، سناریوی نقض زیر لازم است.

۴.۳.۵ سناریوی نقض – برنامه (الف)

دستور اجرا

```
spin -search -ltl noDivByZero verify.pml
```

خروجی ترمینال

```
1 pan: ltl formula noDivByZero
2 pan:1: assertion violated !( !(divByZero))) (at depth 12)
3 pan: wrote verify.pml.trail
4
5 (Spin Version 6.5.1 -- 31 July 2020)
6 Warning: Search not completed
7 + Partial Order Reduction
8
9 Full statespace search for:
10 never claim + (noDivByZero)
11 assertion violations + (if within scope of claim)
12 acceptance cycles + (fairness disabled)
13 invalid end states - (disabled by never claim)
14
15 State-vector 32 byte, depth reached 12, errors: 1
16 7 states, stored
17 0 states, matched
18 7 transitions (= stored+matched)
19 0 atomic steps
20 hash conflicts: 0 (resolved)
21
22 pan: elapsed time 0.002 seconds
23
```

تفسیر

در این برنامه $y=0$ از ابتدا مقداردهی شده، پس شاخه‌ی گارد بلافاصله فعال می‌شود. SPIN در عمق ۱۲ (تنها ۷ حالت) یک نقض واقعی برای `noDivByZero` پیدا کرد (errors: 1)، یعنی `divByZero` به `true` تغییر کرده است. این نتیجه دقیقاً همان چیزی است که انتظار می‌رفت: وقتی مسیر خطرناک واقعاً `reachable` است، SPIN آن را تشخیص می‌دهد. مجموع دو سناریو نشان می‌دهد که فلگ `divByZero` در مدل پایه به این دلیل هرگز `true` نشد که مسیر خطا در آن برنامه‌ی خاص وجود نداشت، نه به این دلیل که مکانیزم گارد ناقص است.

۴.۵ خصوصیت دوم – Liveness

۱.۴.۵ فرمول LTL

$$\Box (inLoop \rightarrow \Diamond exitLoop)$$

معادل آن در `properties.ltl`: `ltl loopExit: [] ((! (inLoop)) || (<> (exitLoop)))`

۲.۴.۵ چرا فرمول به این شکل است؟

خصوصیت‌های Liveness معمولاً به صورت $\Box(P \rightarrow \Diamond Q)$ بیان می‌شوند. مفهوم این فرمول آن است که هرگاه شرط P برقرار شود، سیستم موظف است در ادامه اجرای خود نهایتاً به حالتی برسد که Q برقرار باشد.

در پروژه حاضر، پس از ورود به یک حلقه، انتظار می‌رود که اجرای برنامه در نهایت از آن حلقه خارج شود. به همین دلیل از فرمول

$$\Box(inLoop \rightarrow \Diamond exitLoop)$$

استفاده شده است.

اگر تنها از عملگر $\Diamond$ استفاده می‌شد، مانند

$$\Diamond(exitLoop)$$

این فرمول فقط بیان می‌کرد که «در یکی از مسیرهای اجرا ممکن است از حلقه خارج شویم» و هیچ تضمینی درباره تمام دفعات ورود به حلقه ارائه نمی‌داد. استفاده همزمان از $\Box$ و $\Diamond$ تضمین می‌کند که خروج از حلقه برای تمام دفعات ورود به آن در نهایت رخ خواهد داد.

۳.۴.۵ سناریوی تایید – مدل پایه

دستور اجرا

```
spin -search -ltl loopExit verify.pml
```

خروجی ترمینال

```
1 (Spin Version 6.5.1 -- 31 July 2020)
2 + Partial Order Reduction
3
4 Full statespace search for:
5 never claim + (loopExit)
6 assertion violations + (if within scope of claim)
7 acceptance cycles + (fairness disabled)
8 invalid end states - (disabled by never claim)
9
10 State-vector 48 byte, depth reached 137, errors: 0
11 110 states, stored (151 visited)
12 77 states, matched
13 228 transitions (= visited+matched)
14 0 atomic steps
15 hash conflicts: 0 (resolved)
16
17 unreachable in proctype main
18 output.pml:20, state 7, "divByZero = 1"
19 output.pml:21, state 8, "errFlag_1 = 1"
20 output.pml:29, state 15, "safe = 0"
21 (6 of 69 states)
```

```

22      unreachable in claim loopExit
23      _spin_nvr.tmp:19, state 13, "-end-"
24      (1 of 13 states)
25
26      pan: elapsed time 0.003 seconds
27

```

تفسیر

هیچ اجرایی پیدا نشد که در آن `inLoop` برقرار شود ولی `exitLoop` هرگز به دنبالش نیاید (errors: 0). چون هر دو حلقه‌ی مدل پایه ($i < 5$ و $j < 3$) کراندارند و همیشه با `break` خاتمه می‌یابند، این نتیجه تا حدی بدیهی است: در این برنامه‌ی خاص اساساً امکان ورود به یک حلقه‌ی نامتناهی وجود ندارد. برای بررسی غیر بدیهی، سناریوی نقض زیر با برنامه (ج) طراحی شد.

۴.۴.۵ سناریوی نقض (تلاش) – برنامه (ج-۱)

دستور اجرا

```
spin -search -ltl loopExit example/liveness.pml
```

خروجی ترمینال

```

1      pan: ltl formula loopExit
2      error: max search depth too small
3
4      (Spin Version 6.5.1 -- 31 July 2020)
5      + Partial Order Reduction
6
7      Full statespace search for:
8      never claim          + (loopExit)
9      assertion violations + (if within scope of claim)
10     acceptance cycles    + (fairness disabled)
11     invalid end states    - (disabled by never claim)
12
13     State-vector 44 byte, depth reached 9999, errors: 0
14     9992 states, stored (14984 visited)
15     9986 states, matched
16     24970 transitions (= visited+matched)
17     0 atomic steps
18     hash conflicts: 1 (resolved)
19
20     unreachable in proctype main
21     example/liveness.pml:26, state 15, "activeLoopCount = (
activeLoopCount-1)"
22     example/liveness.pml:36, state 25, "activeLoopCount = (
activeLoopCount+1)"
23     example/liveness.pml:41, state 29, "safe = (safe+j)"

```

```

24 example/liveness.pml:54, state 44, "printf('%d\n',safe)"
25 example/liveness.pml:56, state 45, "endReached = 1"
26 example/liveness.pml:58, state 47, "-end-"
27 (19 of 47 states)
28 unreachable in claim loopExit
29 _spin_nvr.tmp:19, state 13, "-end-"
30 (1 of 13 states)
31
32 pan: elapsed time 0.011 seconds
33 pan: rate 1362181.8 states/second
34

```

تفسیر

این اجرا با پیغام error: max search depth too small یعنی عمق جستجوی پیش فرض SPIN (پارامتر -m، با مقدار پیش فرض ۹۹۹۹) قبل از تکمیل جستجوی تودرتو (nested DFS) برای یافتن accepting cycle تمام شد. بنابراین errors: 0 در اینجا به هیچ وجه به معنای تأیید خصوصیت نیست؛ نتیجه صرفاً ناقص است.

با این حال، بخش unreachable شاهد غیرمستقیم قوی‌ای ارائه می‌دهد: تمام دستورات بعد از حلقه‌ی اول (شامل کل حلقه‌ی دوم، printf و برچسب پایانی) در طول ۹۹۹۲ حالت کاوش شده هرگز اجرا نشده‌اند. این کاملاً با طراحی برنامه سازگار است: چون i هرگز افزایش نمی‌یابد، اجرا برای همیشه در حلقه‌ی اول گیر می‌کند و inLoop بدون رسیدن متناظر exitLoop برقرار می‌ماند. دلیل فنی رشد بزرگ فضای حالت (۹۹۹۲ حالت به جای یک self-loop کوچک) هم قابل توضیح است: در بدنه‌ی حلقه در این برنامه، دستور $x = x - 1$ در هر تکرار اجرا می‌شود، پس علاوه بر نامتناهی بودن خود حلقه، مقدار x هم هرگز تکرار نمی‌شود (هر تکرار یک state برداری جدید تولید می‌کند، چون x جزئی از بردار حالت است). در نتیجه SPIN مجبور است پیش از رسیدن به یک حالت تکراری، مسیر رو به رشدی از حالت‌های یکتا را کاوش کند که در عمل به سقف عمق پیش فرض (-m) 9999 برخورد می‌کند. برای دریافت یک counterexample رسمی، لازم است جستجو با عمق بزرگ‌تر تکرار شود؛ این مورد به عنوان محدودیت شناخته شده و کار آتی ثبت می‌شود.

۵.۵ خصوصیت سوم – Reachability

۱.۵.۵ فرمول LTL

فرمولی که به SPIN داده می‌شود، نقیض فرمول اصلی است:

$$\neg(\Diamond \text{endReached})$$

معادل آن در properties.ltl: `ltl reachEnd: ! (<> (endReached))` نکته‌ی تفسیری مهم: چون فرمول به صورت نقیض داده می‌شود، برخلاف بقیه‌ی خصوصیات، در این مورد یافتن یک نقض (errors: 1) به معنای تأیید reachability است (یعنی مسیری به endReached واقعاً وجود دارد)، و عدم یافتن نقض (errors: 0) به معنای آن است که endReached در آن مدل خاص غیرقابل دسترس است.

۲.۵.۵ چرا فرمول به این شکل است؟

در SPIN معمولاً خصوصیت‌های دسترس‌پذیری به صورت مستقیم بیان نمی‌شوند، زیرا LTL ذاتاً برای بیان خواصی که باید در تمام مسیرهای اجرا برقرار باشند طراحی شده است. برای بررسی اینکه آیا حالت پایانی برنامه قابل دسترسی است، نقیض این ویژگی نوشته می‌شود:

$$\neg \Diamond(endReached)$$

در صورت نقض این فرمول توسط SPIN، یک مسیر نمونه ارائه می‌شود که در آن متغیر endReached فعال شده است. بنابراین شکست این فرمول به معنای آن است که حالت پایانی برنامه واقعاً قابل دستیابی بوده و ویژگی Reachability برقرار است. به همین دلیل در این پروژه از نقیض ویژگی دسترس‌پذیری استفاده شده است.

۳.۵.۵ سناریوی تایید – مدل پایه (انتظار: نقض یافت شود)

دستور اجرا

```
spin -search -ltl reachEnd verify.pml
```

خروجی ترمینال

```
1 pan: ltl formula reachEnd
2 pan:1: assertion violated !(endReached) (at depth 132)
3 pan: wrote verify.pml.trail
4
5 (Spin Version 6.5.1 -- 31 July 2020)
6 Warning: Search not completed
7 + Partial Order Reduction
8
9 Full statespace search for:
10 never claim + (reachEnd)
11 assertion violations + (if within scope of claim)
12 acceptance cycles + (fairness disabled)
13 invalid end states - (disabled by never claim)
14
15 State-vector 48 byte, depth reached 132, errors: 1
16 67 states, stored
17 0 states, matched
18 67 transitions (= stored+matched)
19 0 atomic steps
20 hash conflicts: 0 (resolved)
21
22 pan: elapsed time 0.003 seconds
23
```

تفسیر

SPIN در عمق ۱۳۲ یک نقض برای فرمول نقیض شده پیدا کرد (errors: 1)؛ یعنی مسیر اجرایی یافت که در آن endReached برقرار می‌شود. طبق نکته‌ی بالا، این نتیجه دقیقاً همان چیزی است که انتظار می‌رفت: مدل پایه یک برنامه‌ی خطی و deterministic است که همیشه به انتهای خود می‌رسد، پس برچسب پایانی حتماً باید reachable باشد. یافتن این counterexample تأیید می‌کند که خصوصیت Reachability برقرار است.

۴.۵.۵ سناریوی مکمل – برنامه (ج-۲) (انتظار: نقض یافت نشود)

برای اینکه نشان دهیم SPIN می‌تواند بین «قابل دسترس» و «غیرقابل دسترس» تمایز واقعی قائل شود (نه اینکه صرفاً همیشه یک counterexample پیدا کند)، همین بررسی روی برنامه (ج-۲) که حلقه‌ی اولش نامتناهی است تکرار شد. توجه شود که این برنامه با برنامه (ج-۱) که برای Liveness استفاده شد متفاوت است؛ در (ج-۲) بدنه‌ی حلقه هیچ متغیری را تغییر نمی‌دهد (safe = safe)، به همین دلیل state برنامه خیلی زود تکرار می‌شود و جستجو، برخلاف (ج-۱)، به‌طور کامل و بدون برخورد به سقف عمق به پایان می‌رسد.

دستور اجرا

```
spin -search -ltl reachEnd example/reachability.pml
```

خروجی ترمینال

```
1 pan: ltl formula reachEnd
2
3 (Spin Version 6.5.1 -- 31 July 2020)
4 + Partial Order Reduction
5
6 Full statespace search for:
7 never claim + (reachEnd)
8 assertion violations + (if within scope of claim)
9 acceptance cycles + (fairness disabled)
10 invalid end states - (disabled by never claim)
11
12 State-vector 44 byte, depth reached 19, errors: 0
13 10 states, stored
14 1 states, matched
15 11 transitions (= stored+matched)
16 0 atomic steps
17 hash conflicts: 0 (resolved)
18
19 unreached in proctype main
20 example/reachability.pml:24, state 14, "activeLoopCount = (
activeLoopCount-1)"
21 example/reachability.pml:39, state 28, "safe = (safe+j)"
22 example/reachability.pml:52, state 43, "printf('%d\n',safe)"
23 example/reachability.pml:54, state 44, "endReached = 1"
```

```

24 example/reachability.pml:56, state 46, "-end-"
25 (19 of 46 states)
26 unreachable in claim reachEnd
27 _spin_nvr.tmp:28, state 10, "-end-"
28 (1 of 10 states)
29
30 pan: elapsed time 0.006 seconds
31

```

تفسیر

این بار جستجو به طور کامل تمام شد (بدون خطای عمق، چون خود فرمول reachEnd با فضای حالت کوچکتری قابل رد کردن است) و هیچ نقضی یافت نشد (errors: 0). بخش unreachable به وضوح نشان می‌دهد خط $\text{endReached} = 1$ هرگز اجرا نشده است. این با تحلیل دستی برنامه کاملاً سازگار است: چون حلقه‌ی اول هرگز پایان نمی‌یابد، اجرا هرگز به بخش دوم برنامه یا برچسب پایانی نمی‌رسد. مقایسه‌ی این نتیجه با سناریوی قبلی (مدل پایه) نشان می‌دهد SPIN واقعاً بین قابل دسترس بودن و نبودن endReached فرق می‌گذارد، نه اینکه نتیجه‌اش وابسته به یک الگوی ثابت باشد.

۶.۵ خصوصیت چهارم – Deadlock Freedom

این خصوصیت نیازی به فرمول LTL ندارد و با قابلیت توکار SPIN بررسی می‌شود. برخلاف چهار خصوصیت دیگر، اینجا سناریوی نقض طراحی نشده است؛ دلیل آن در پایان این بخش توضیح داده شده.

۱.۶.۵ دستور اجرا

```

1 spin -search -deadlock verify.pml
2

```

۲.۶.۵ خروجی ترمینال (مسیر اجرای کامل)

از آنجا که مدل پایه کاملاً deterministic است، SPIN تنها یک مسیر اجرایی یکتا برای proctype main می‌یابد:

```

1 proctype main
2   state 1 -> state 2   verify.pml:13 x = 10
3   state 2 -> state 3   verify.pml:14 y = 2
4   state 3 -> state 4   verify.pml:15 i = 0
5   state 4 -> state 5   verify.pml:16 safe = 0
6   state 5 -> state 12  verify.pml:18 errFlag_1 = 0
7   state 12 -> state 11 verify.pml:19 else (y != 0)
8   state 11 -> state 18 verify.pml:24 safe = (x / y)
9   state 18 -> state 17 verify.pml:28 else (!errFlag_1)
10  state 20 -> state 28 verify.pml:32 activeLoopCount = activeLoopCount + 1
11  state 28 -> state 22 verify.pml:35 (i < 5)
12  state 28 -> state 31 verify.pml:35 else (i >= 5)
13  state 39 -> state 41 verify.pml:51 j = 0
14  state 41 -> state 49 verify.pml:52 activeLoopCount = activeLoopCount + 1
15  state 49 -> state 43 verify.pml:54 (j < 3)
16  state 49 -> state 52 verify.pml:54 else (j >= 3)
17  state 64 -> state 61 verify.pml:71 (x >= 0)

```

```

18 state 61 -> state 66 verify.pml:72 safe = safe + 1
19 state 66 -> state 67 verify.pml:76 printf('%d\n', safe)
20 state 67 -> state 68 verify.pml:78 endReached = 1
21 state 68 -> state 69 verify.pml:79 (1)
22 state 69 -> state 0 verify.pml:80 -end-
23

```

۳.۶.۵ تفسیر

مسیر اجرا بدون هیچ توقف غیرمنتظره‌ای به برچسب -end- می‌رسد که یک پایان معتبر (valid end) state است؛ بنابراین مدل عاری از deadlock است.

چرا سناریوی نقض طراحی نشد؟ طبق قوانین ترجمه‌ی پیاده‌سازی‌شده در ترنسلیتور، هر if/ff تولیدشده (چه از age/vagarna، چه از گاردهای تقسیم، چه از چک errFlag) به‌صورت خودکار یک شاخه‌ی skip -> else دارد، و هر do/od هم شاخه‌ی break -> else صریح دارد. بنابراین به‌صورت ساختاری هیچ حالتی در خروجی این ترنسلیتور وجود ندارد که در آن هیچ transition ای فعال نباشد؛ عاری‌بودن از deadlock نتیجه‌ی مستقیم این تصمیم طراحی است، نه یک ویژگی تصادفی که فقط با شانس در یک تست خاص برقرار شده باشد. تولید یک نقض واقعی مستلزم دستکاری دستی خروجی Promela (مثلاً حذف عمدی یک شاخه‌ی else) است که دیگر محصول ترنسلیتور نیست، بلکه یک Promela دستی و مصنوعی خواهد بود؛ به همین دلیل چنین سناریویی در این گزارش گنجانده نشد.

۷.۵ خصوصیت پنجم – Invariant

۱.۷.۵ فرمول LTL

$$\Box (x \geq 0)$$

معادل آن در properties.ltl: `ltl nonNegativeX: [] ((x>=0))`

۲.۷.۵ چرا فرمول به این شکل است؟

Invariant یا خصوصیتی است که باید در تمام طول اجرای سیستم برقرار بماند. چنین ویژگی‌هایی نیز همانند خصوصیت‌های ایمنی با عملگر $\Box$ بیان می‌شوند. در این پروژه فرض شده است که مقدار متغیر x نباید هیچ‌گاه منفی شود؛ بنابراین فرمول زیر استفاده شده است:

$$\Box (x \geq 0)$$

اگر تنها از عملگر $\Diamond$ استفاده می‌شد، فرمول صرفاً بیان می‌کرد که در یکی از حالت‌های اجرا مقدار متغیر غیرمنفی است، در حالی که هدف invariant تضمین برقرار بودن این شرط در تمام حالات قابل دسترس است.

۳.۷.۵ سناریوی تایید – مدل پایه

دستور اجرا

```
spin -search -ltl nonNegativeX verify.pml
```

```

1      (Spin Version 6.5.1 -- 31 July 2020)
2      + Partial Order Reduction
3
4      Full statespace search for:
5      never claim          + (nonNegativeX)
6      assertion violations + (if within scope of claim)
7      acceptance cycles   + (fairness disabled)
8      invalid end states  - (disabled by never claim)
9
10     State-vector 48 byte, depth reached 137, errors: 0
11     69 states, stored
12     1 states, matched
13     70 transitions (= stored+matched)
14     0 atomic steps
15     hash conflicts: 0 (resolved)
16
17     unreached in proctype main
18     output.pml:20, state 7, "divByZero = 1"
19     output.pml:21, state 8, "errFlag_1 = 1"
20     output.pml:29, state 15, "safe = 0"
21     (6 of 69 states)
22     unreached in claim nonNegativeX
23     _spin_nvr.tmp:37, state 10, "-end-"
24     (1 of 10 states)
25
26     pan: elapsed time 0.005 seconds
27

```

تفسیر

جستجوی کامل هیچ حالتی با $x < 0$ پیدا نکرد (errors: 0). این نتیجه، مثل خصوصیت Safety، تا حدی بدیهی است: متغیر x از مقدار اولیه ۱۰ شروع می‌شود و تنها در حلقه‌ی اول، دقیقاً ۵ بار یک واحد کاهش می‌یابد؛ بنابراین کمترین مقدار ممکن برای x در این برنامه‌ی خاص برابر ۵ است و ساختاراً هرگز امکان منفی‌شدن وجود ندارد.

۴.۷.۵ سناریوی نقض – برنامه (ب)

دستور اجرا

```
spin -search -ltl nonNegativeX verify.pml
```

```

1      pan: ltl formula nonNegativeX
2      pan:1: assertion violated !((x>=0))) (at depth 44)
3      pan: wrote verify.pml.trail
4
5      (Spin Version 6.5.1 -- 31 July 2020)
6      Warning: Search not completed
7      + Partial Order Reduction
8
9      Full statespace search for:
10     never claim          + (nonNegativeX)
11     assertion violations  + (if within scope of claim)
12     acceptance  cycles   + (fairness disabled)
13     invalid end states   - (disabled by never claim)
14
15     State-vector 36 byte, depth reached 44, errors: 1
16     23 states, stored
17     0 states, matched
18     23 transitions (= stored+matched)
19     0 atomic steps
20     hash conflicts: 0 (resolved)
21
22     pan: elapsed time 0.006 seconds
23

```

تفسیر

در این برنامه x از ۳ شروع می‌شود و در طول ۱۰ تکرار حلقه ($i < 10$) به‌طور پیوسته کاهش می‌یابد. SPIN در عمق ۴۴ یک نقض واقعی پیدا کرد (errors: 1)، یعنی مقدار x در جایی از اجرا منفی شده است. این تأیید می‌کند که فرمول $\square(x \geq 0)$ واقعاً توسط SPIN روی مقادیر پویا بررسی می‌شود، نه اینکه نتیجه‌ی مدل پایه به این دلیل بود که invariant همیشه برقرار است.

فاز سوم

۸.۵ مقدمه فاز سوم

در فاز دوم، پنج خصوصیت مشخص را با استفاده از SPIN روی چند برنامه‌ی مشخص و متناهی Hash بررسی کردیم. نتیجه‌ی هر بار اجرای SPIN تنها درباره‌ی همان یک برنامه‌ی خاص حرف می‌زد؛ برای مثال، خصوصیت Invariant (nonNegativeX) فقط ثابت کرد که در مدل پایه‌ی مشخص فاز اول، متغیر x هرگز منفی نمی‌شود – نه اینکه در هر برنامه‌ی Hash که از الگوی مشابهی پیروی کند، همین اتفاق می‌افتد.

هدف فاز سوم رفتن از این نتیجه‌ی موردی به یک تضمین کلی و ریاضی است: می‌خواهیم ثابت کنیم که اگر یک شرط invariant قبل از شروع یک حلقه برقرار باشد و بدنه‌ی حلقه آن را حفظ کند، آنگاه بعد از پایان کامل حلقه – برای هر برنامه، هر شرط حلقه، و هر تعداد تکرار – آن invariant همچنان برقرار خواهد بود.

برای بیان چنین ادعای کلی‌ای، برخلاف فاز دوم که یک مدل متناهی کافی بود، به یک تعریف **صوری و کامل** از معنای اجرای هر دستور Hash نیاز داریم؛ یعنی Operational Semantics. سپس ادعا را به عنوان یک قضیه‌ی ریاضی درباره‌ی این تعریف بیان و در Lean 4 اثبات می‌کنیم.

۹.۵ اثبات ریاضی قضیه با استقرا روی اشتقاق

پیش از رفتن سراغ کد Lean، ابتدا قضیه را به‌طور کاملاً مستقل و روی کاغذ، با استقرا روی ساختار اشتقاق (derivation) اثبات می‌کنیم. همین منطق در بخش بعد، خطبه‌خط به کد Lean ترجمه می‌شود.

۱.۹.۵ بیان دقیق قضیه

فرض کنید P یک گزاره (predicate) روی حالت‌هاست. ادعا می‌کنیم:

$$P(\sigma) \wedge (\forall \sigma_1 \sigma_2, P(\sigma_1) \wedge \langle b, \sigma_1 \rangle \Downarrow \text{dorost} \wedge \langle S, \sigma_1 \rangle \Downarrow \sigma_2 \Rightarrow P(\sigma_2)) \wedge (\langle \text{ta}(b)\{S\}, \sigma \rangle \Downarrow \sigma' \Rightarrow P(\sigma'))$$

یعنی: اگر P قبل از حلقه برقرار باشد و بدنه‌ی حلقه آن را حفظ کند، آنگاه در هر اشتقاقی که $\langle \text{ta}(b)\{S\}, \sigma \rangle \Downarrow \sigma'$ را نتیجه بدهد، $P(\sigma')$ برقرار است.

۲.۹.۵ چرا استقرا روی اشتقاق، نه روی عدد تکرار؟

نکته‌ی مهم این است که در تعریف Big-Step، هیچ‌جا عدد تکرار حلقه به‌صراحت شمرده نمی‌شود. آنچه داریم فقط یک **درخت اشتقاق** است که فرضیه‌ی $\langle \text{ta}(b)\{S\}, \sigma \rangle \Downarrow \sigma'$ را نتیجه می‌دهد. طبق قوانین استنتاجی سند پروژه، تنها دو قانون وجود دارد که نتیجه‌شان می‌تواند به این شکل باشد:

While-F و While-T (چهار قانون دیگر – If-F, If-T, Seq, Assign – نتیجه‌شان همیشه دستوری غیر از ta است و اصلاً نمی‌توانند این حکم را تولید کنند). پس اثبات با استقرا روی ساختار همین اشتقاق انجام می‌شود: دقیقاً دو حالت داریم که باید بررسی شوند – یکی به‌عنوان پایه‌ی استقرا و دیگری به‌عنوان گام استقرا.

۳.۹.۵ پایه‌ی استقرا – قانون While-F

فرض کنید آخرین قانون استفاده‌شده در اشتقاق، While-F باشد:

$$\frac{\langle b, \sigma \rangle \Downarrow ghalat}{\langle ta(b)\{S\}, \sigma \rangle \Downarrow \sigma} \text{ (While-F)}$$

طبق این قانون، حالت پایانی دقیقاً همان حالت شروع است، یعنی $\sigma' = \sigma$. پس باید نشان دهیم $P(\sigma')$ برقرار است، که با جای‌گذاری برابر است با نشان‌دادن $P(\sigma)$. اما این دقیقاً همان مقدمه‌ی اول قضیه است که از پیش داریم:

$$P(\sigma) \Rightarrow P(\sigma') \quad (\text{چون } \sigma' = \sigma)$$

پس پایه‌ی استقرا بدون نیاز به هیچ استدلال اضافه‌ای برقرار است.

۴.۹.۵ گام استقرا – قانون While-T

فرض کنید آخرین قانون استفاده‌شده، While-T باشد:

$$\frac{\langle b, \sigma \rangle \Downarrow dorost \quad \langle S, \sigma \rangle \Downarrow \sigma'' \quad \langle ta(b)\{S\}, \sigma'' \rangle \Downarrow \sigma'}{\langle ta(b)\{S\}, \sigma \rangle \Downarrow \sigma'} \text{ (While-T)}$$

این قانون سه مقدمه دارد. سومین مقدمه، یعنی $\langle ta(b)\{S\}, \sigma'' \rangle \Downarrow \sigma'$ ، خودش یک اشتقاق کامل و مستقل برای همان حکم $ta(b)S$ است، با این تفاوت که حالت شروعش σ'' است نه σ ؛ و چون این زیر-اشتقاق زیرمجموعه‌ای از اشتقاق اصلی است، فرض استقرا (IH) برایش صدق می‌کند:

$$(IH) \quad P(\sigma'') \wedge \langle ta(b)\{S\}, \sigma'' \rangle \Downarrow \sigma' \Rightarrow P(\sigma')$$

پس کافی است نشان دهیم $P(\sigma'')$ برقرار است؛ آنگاه با اعمال IH، مستقیماً $P(\sigma')$ نتیجه می‌شود. برای اثبات $P(\sigma'')$ ، از سه چیزی که در این حالت در دسترس داریم استفاده می‌کنیم:

• $P(\sigma)$ (فرض قضیه، چون هنوز روی حالت شروع σ هستیم)،

• $\langle b, \sigma \rangle \Downarrow dorost$ (اولین مقدمه‌ی While-T)،

• $\langle S, \sigma \rangle \Downarrow \sigma''$ (دومین مقدمه‌ی While-T).

که دقیقاً سه شرط مقدمه‌ی دوم قضیه (پایستگی P توسط بدنه) هستند. با اعمال مستقیم آن مقدمه:

$$P(\sigma) \wedge \langle b, \sigma \rangle \Downarrow dorost \wedge \langle S, \sigma \rangle \Downarrow \sigma'' \Rightarrow P(\sigma'')$$

نتیجه می‌گیریم $P(\sigma'')$. حال با جای‌گذاری در IH بالا:

$$P(\sigma'') \Rightarrow P(\sigma')$$

و در نهایت $P(\sigma')$ اثبات می‌شود.

۵.۹.۵ جمع‌بندی اثبات

چون هر اشتقاق $\sigma' \Downarrow \langle ta(b)\{S\}, \sigma \rangle$ فقط می‌تواند با یکی از دو قانون While-F یا While-T ساخته شده باشد، و در هر دو حالت نشان دادیم $P(\sigma')$ برقرار است (در پایه به‌طور مستقیم، در گام با کمک فرض استقرا)، طبق اصل استقرا روی ساختار اشتقاق، قضیه برای هر اشتقاق ممکن از $\langle ta(b)\{S\}, \sigma \rangle \Downarrow \sigma'$ درست است.

پیوند با کد Lean

این دقیقاً همان دو شاخه‌ای است که در تابع aux بخش بعد، پس از باز کردن cases hEq، تحت نام‌های whileTrue و whileFalse ظاهر می‌شوند: شاخه‌ی whileFalse معادل «پایه‌ی استقرا» بالاست (exact hInv دقیقاً همان استفاده از $P(\sigma)$ به جای $P(\sigma')$ است)، و شاخه‌ی whileTrue معادل «گام استقرا» است (bodyPreservesInvariant همان اعمال مقدمه‌ی دوم برای رسیدن به $P(\sigma'')$ است، و ihRest همان استفاده از فرض استقرا IH روی زیر-اشتقاق کوچکتر است). چهار قانون دیگر (Seq، Assign، If-F، If-T) هم در اثبات ریاضی بالا اصلاً ظاهر نشدند، دقیقاً به همان دلیلی که در کد Lean با cases hEq بلافاصله رد می‌شوند: نتیجه‌ی آن‌ها هرگز از شکل $\langle ta(b)\{S\}, \sigma \rangle \Downarrow \sigma'$ نیست.

۱۰.۵ بازنگری مفاهیم کلیدی

۱.۱۰.۵ Big-Step

در این پروژه از رویکرد Big-Step (یا Natural Semantics) استفاده شده است. نماد اصلی آن:

$$\langle S, \sigma \rangle \Downarrow \sigma'$$

خوانده می‌شود: «دستور S در محیط σ اجرا می‌شود و به محیط σ' می‌رسد.» برخلاف Small-Step که هر قدم اجرا را جداگانه نشان می‌دهد، Big-Step فقط رابطه‌ی بین حالت شروع و حالت پایان کامل اجرا را بیان می‌کند – دقیقاً همان چیزی که برای اثبات این قضیه لازم است.

توجه:

در فایل Lean، BigStep به صورت یک Prop استقرایی سه‌آرگومانی تعریف شده، نه یک تابع. دلیل این انتخاب این است که اجرای یک حلقه‌ی ta (while) ممکن است برای برخی ورودی‌ها هرگز خاتمه نیابد؛ یک رابطه به‌سادگی اجازه می‌دهد در چنین حالتی هیچ σ' ای با S و σ در رابطه نباشد، بدون نیاز به تعریف یک «مقدار خطا» یا «مقدار بی‌نهایت» مصنوعی.

۱۱.۵ تعریف انواع پایه در Lean

اولین گام، تعریف صوری نحو (syntax) زیرمجموعه‌ی Hash است – دقیقاً همان چیزی که Parser فاز اول از روی گرامر Hash.g4 می‌ساخت، اما این بار به شکل یک نوع داده‌ی Lean:

```
1 abbrev Var := String
2 abbrev State := Var -> Int
3
4 inductive Expr where
5   | num : Int -> Expr
```

```

6 | var : Var -> Expr
7 | add : Expr -> Expr -> Expr
8 | mul : Expr -> Expr -> Expr
9 | lt : Expr -> Expr -> Expr
10 | equal : Expr -> Expr -> Expr
11
12 inductive Stmt where
13 | assign : Var -> Expr -> Stmt
14 | seq : Stmt -> Stmt -> Stmt
15 | ifElse : Expr -> Stmt -> Stmt -> Stmt
16 | while : Expr -> Stmt -> Stmt
17

```

• State همان σ در فرمول‌های ریاضی است: یک تابع از نام متغیر به مقدار صحیح آن. هیچ نوع جداگانه‌ای برای Bool تعریف نشده؛ مقادیر بولی زبان Hash (ghalat/dorost) با اعداد صحیح شبیه‌سازی می‌شوند (غیرصفر = درست، صفر = نادرست) – همان الگویی که در فاز اول برای ترجمه به Promela هم استفاده شد.

• Expr علاوه بر چهار سازنده‌ی مثال راهنما (lt, add, var, num)، دو سازنده‌ی mul و equal هم اضافه شده تا زیرمجموعه‌ی پوشش داده شده به عملگرهای محاسباتی و مقایسه‌ای فاز اول نزدیک‌تر باشد.

• Stmt همان چهار دستور اصلی مثال راهنما را دارد (while, ifElse, seq, assign) که برای اثبات این قضیه کافی‌اند؛ ساختارهای edame/shekan, baraye و emtehan/gereftar طبق صورت پروژه لازم نیست در این فاز صورتی‌سازی شوند.

۱۲.۵ ارزیابی عبارات و به‌روزرسانی State

```

1 def evalExpr (e : Expr) (s : State) : Int :=
2   match e with
3   | Expr.num n => n
4   | Expr.var x => s x
5   | Expr.add e1 e2 => evalExpr e1 s + evalExpr e2 s
6   | Expr.mul e1 e2 => evalExpr e1 s * evalExpr e2 s
7   | Expr.lt e1 e2 =>
8     if evalExpr e1 s < evalExpr e2 s then 1 else 0
9   | Expr.equal e1 e2 =>
10    if evalExpr e1 s = evalExpr e2 s then 1 else 0
11
12 def update (s : State) (x : Var) (v : Int) : State :=
13   fun y => if y = x then v else s y
14

```

evalExpr به‌جای اینکه (مثل قوانین استنتاجی سند پروژه) به شکل یک رابطه‌ی جداگانه با قوانین Add, Var, Num و ... تعریف شود، مستقیماً به صورت یک تابع بازگشتی نوشته شده است. برخلاف BigStep، ارزیابی عبارات (Expr) هیچ‌وقت با مشکل عدم‌خاتمه روبه‌رو نیست – هر عبارت، هرچقدر هم تودرتو، در یک تعداد قدم محدود و ثابت به یک مقدار می‌رسد و هیچ دو مسیر اجرای متفاوتی هم برایش وجود ندارد. به همین دلیل نوشتن evalExpr به‌صورت یک تابع معمولی (نه یک رابطه‌ی استقرایی مثل BigStep) کاملاً بلامانع است. این تابع همان چیزی را می‌گوید که قوانین استنتاجی سند

پروژه (Add، Var، Num و ...) می‌گفتند، با یک مزیت اضافه: چون خروجی تابع همیشه به‌خودی‌خود یکتاست (طبق تعریف خود تابع در Lean)، دیگر لازم نیست قطعیت عبارات را جداگانه اثبات کنیم – چیزی که اگر عبارات را هم مثل دستورات با یک رابطه تعریف می‌کردیم، لازم می‌شد (همان گزینه‌ی اول صورت پروژه: Expression Determinism).
 update همان نماد $\sigma[x \mapsto v]$ است: یک تابع (State) جدید که با s یکسان است مگر روی x، که مقدار v را می‌گیرد.

۱۳.۵ رابطه‌ی BigStep

```

1  inductive BigStep : Stmt -> State -> State -> Prop where
2
3  | assign {x : Var} {e : Expr} {s : State} :
4  BigStep
5  (Stmt.assign x e)
6  s
7  (update s x (evalExpr e s))
8
9  | seq {S1 S2 : Stmt} {s sMid sFinal : State} :
10 BigStep S1 s sMid ->
11 BigStep S2 sMid sFinal ->
12 BigStep (Stmt.seq S1 S2) s sFinal
13
14 | ifTrue {b : Expr} {S1 S2 : Stmt} {s sFinal : State} :
15 evalExpr b s = 0 ->
16 BigStep S1 s sFinal ->
17 BigStep (Stmt.ifElse b S1 S2) s sFinal
18
19 | ifFalse {b : Expr} {S1 S2 : Stmt} {s sFinal : State} :
20 evalExpr b s = 0 ->
21 BigStep S2 s sFinal ->
22 BigStep (Stmt.ifElse b S1 S2) s sFinal
23
24 | whileFalse {b : Expr} {body : Stmt} {s : State} :
25 evalExpr b s = 0 ->
26 BigStep (Stmt.while b body) s s
27
28 | whileTrue {b : Expr} {body : Stmt} {s sMid sFinal : State} :
29 evalExpr b s = 0 ->
30 BigStep body s sMid ->
31 BigStep (Stmt.while b body) sMid sFinal ->
32 BigStep (Stmt.while b body) s sFinal
33

```

هر سازنده (constructor) این نوع استقرایی دقیقاً معادل یکی از قوانین استنتاجی ذکرشده در سند پروژه است (Assign، Seq، If-F/If-T، While-T/While-F). تنها تفاوت نحوی نسبت به مثال راهنمای determinism این است که به‌جای $\text{forall } x \text{ e s, ...}$ از آرگومان‌های ضمنی (implicit) با آکولاد {} استفاده شده است؛ از نظر معنایی هیچ تفاوتی ندارد – Lean مقدار این آرگومان‌ها را خودش از روی زمینه (context) استنتاج می‌کند – و فقط کد را کوتاه‌تر و خواناتر می‌کند.

نکته‌ی مهمی که در سازنده‌ی `whileTrue` دیده می‌شود این است که خودش را در سومین مقدمه صدا می‌زند (`sMid sFinal (BigStep (Stmt.while b body))`)؛ این بازگشتی بودن دقیقاً معنای «حلقه = یک بار اجرای بدنه + ادامه‌ی همان حلقه از حالت جدید» را می‌رساند و پایه‌ی اصلی روش اثبات قضیه در بخش بعد است.

۱۴.۵ قضیه‌ی انتخابی – Invariant Loop

از میان سه گزینه‌ی صورت پروژ، گزینه‌ی دوم (پایستگی ناوردای حلقه) انتخاب و اثبات شده است.

۱.۱۴.۵ صورت رسمی قضیه

$$P(\sigma) \wedge (\forall \sigma_1 \sigma_2, P(\sigma_1) \wedge \langle b, \sigma_1 \rangle \Downarrow \text{dorost} \wedge \langle S, \sigma_1 \rangle \Downarrow \sigma_2 \Rightarrow P(\sigma_2)) \wedge \langle \text{ta}(b) \{S\}, \sigma \rangle \Downarrow \sigma' \Rightarrow P(\sigma')$$

نکته‌ای درباره‌ی سور مقدمه‌ی دوم. صورت پروژ مقدمه‌ی دوم را به شکل فشرده $P(\sigma) \wedge \langle S, \sigma \rangle \Downarrow \sigma' \wedge$ نوشته است، یعنی روی حالت‌های ثابت σ و σ' در فرمول بالا، این مقدمه عمداً با یک سور همگانی $\forall \sigma_1 \sigma_2$ بازنویسی شده است، به این دلیل که اثبات با استقرا روی تعداد دلخواه تکرار حلقه پیش می‌رود: در تکرار دوم به بعد، بدنه‌ی حلقه دیگر از همان σ اولیه شروع نمی‌شود، بلکه از حالت میانی حاصل از تکرار قبلی آغاز می‌گردد. برای این که بتوان «حفظ شدن P توسط بدنه» را در هر یک از این تکرارها – نه فقط تکرار اول – به کار برد، این خاصیت باید برای تمام جفت‌حالت‌های ممکن برقرار فرض شود، نه فقط برای یک جفت خاص. همین سور همگانی است که در کد `Lean` به صورت `forall s s1, ... bodyPreservesInvariant` ظاهر شده و امکان فراخوانی مکرر آن در هر لایه از استقرا (در شاخه‌ی `whileTrue`) را فراهم می‌کند؛ بدون این تعمیم، اثبات تنها برای حلقه‌هایی با دقیقاً یک بار تکرار درست از آب درمی‌آمد. علاوه بر این، شرط $\langle b, \sigma_1 \rangle \Downarrow \text{dorost}$ (که در نگارش فشرده‌ی صورت پروژ نیامده) نیز به صراحت اضافه شده تا مشخص باشد این پایستگی فقط وقتی لازم است که شرط حلقه واقعاً برقرار بوده و بدنه اجرا شده باشد.

به زبان ساده: اگر

۱. ناوردای P قبل از شروع حلقه روی σ برقرار باشد، و

۲. بدنه‌ی حلقه S ، هر بار که با شرط برقرار (b درست) از حالتی که P در آن صادق است اجرا شود، P را حفظ کند،

آنگاه بعد از پایان کامل حلقه‌ی $\text{ta}(b) \{S\}$ (یعنی وقتی شرط سرانجام نادرست شد و حلقه با `whileFalse` تمام شد)، روی حالت نهایی σ' هم برقرار است.

۲.۱۴.۵ چرا این قضیه؟

دلیل انتخاب این گزینه ارتباط مستقیم آن با خصوصیت `Invariant` فاز دوم است. در فاز دوم، با `SPIN` فقط نشان دادیم که در یک برنامه‌ی خاص با یک حلقه‌ی مشخص، متغیر x منفی نمی‌شود – نتیجه‌ای که به درستی در گزارش فاز دوم «تا حدی بدیهی» توصیف شد، چون وابسته به مقادیر اولیه‌ی همان برنامه بود. قضیه‌ی `loopInvariant` این محدودیت را برمی‌دارد: به جای بررسی یک برنامه با `Model Checking`، ثابت می‌کند که اگر بدنه‌ی هر حلقه‌ای طوری نوشته شده باشد که یک شرط را حفظ کند، آنگاه بدون توجه به تعداد تکرار یا مقادیر اولیه، آن شرط بعد از حلقه هم برقرار می‌ماند. این دقیقاً همان قاعده‌ی استنتاج حلقه در منطق هور است و پایه‌ی اثبات صحت هر برنامه‌ی حاوی `while` به شمار می‌رود.

۱۵.۵ کد کامل اثبات و توضیح هر قدم

```
1  theorem loopInvariant
2  {b : Expr}
3  {body : Stmt}
4  {P : State -> Prop}
5  (bodyPreservesInvariant :
6  forall s s1,
7  P s ->
8  evalExpr b s 0 ->
9  BigStep body s s1 ->
10 P s1)
11 :
12 forall s sFinal,
13 P s ->
14 BigStep (Stmt.while b body) s sFinal ->
15 P sFinal := by
16
17 intro s sFinal hP hRun
18
19 have aux :
20 forall {S : Stmt} {s sFinal : State},
21 BigStep S s sFinal ->
22 S = Stmt.while b body ->
23 P s ->
24 P sFinal := by
25
26 intro S s0 s1 h
27 induction h with
28
29 | assign =>
30 intro hEq
31 cases hEq
32
33 | seq hFirst hSecond ihFirst ihSecond =>
34 intro hEq
35 cases hEq
36
37 | ifTrue hCond hThen ih =>
38 intro hEq
39 cases hEq
40
41 | ifFalse hCond hElse ih =>
42 intro hEq
43 cases hEq
44
45 | whileFalse hCond =>
46 intro hEq hInv
47 cases hEq
48 exact hInv
49
```

```

50 | whileTrue hCond hBody hRest ihBody ihRest =>
51 | intro hEq hInv
52 | cases hEq
53 | have hInvAfterBody : P _ :=
54 | bodyPreservesInvariant _ _ hInv hCond hBody
55 | exact ihRest rfl hInvAfterBody
56
57 | exact aux hRun rfl hP
58

```

۱.۱۵.۵ صورت‌بندی قضیه و ورود مقدمات

سه آرگومان ضمنی $body$ ، P ، و یک فرض اصلی به نام $bodyPreservesInvariant$ گرفته می‌شوند که دقیقاً مقدمه‌ی شماره‌ی ۲ از صورت ریاضی بالا را بیان می‌کند: «برای هر دو حالت s و s_1 ، اگر $P(s)$ برقرار باشد، شرط حلقه روی s درست باشد، و بدنه از s به s_1 برسد، آنگاه $P(s_1)$ ». خط $intro s sFinal$ هم بقیه‌ی مقدمات (حالت اولیه، حالت نهایی، برقراری P در ابتدا، و خود اشتقاق اجرای حلقه) را وارد می‌کند.

۲.۱۵.۵ چرا یک لم کمکی (aux) لازم است؟

اینجا مهم‌ترین تفاوت این اثبات با مثال راهنمای $determinism$ است. در مثال راهنما، فرض $h1$ روی $BigStep S s s'$ یک متغیر عمومی S بود، پس می‌شد مستقیماً $h1$ induction زد. اما اینجا فرض $hRun$ روی یک دستور خاص و ثابت، یعنی دقیقاً $Stmt.while b body$ ، است. اگر بخواهیم مستقیم روی $hRun$ استقرا بزنیم، Lean برای هر یک از شش سازنده‌ی $BigStep$ (نه فقط $whileFalse$ و $whileTrue$) یک شاخه می‌سازد، اما نمی‌تواند به‌طور خودکار متوجه شود که آرگومان اول در چهار سازنده‌ی دیگر (seq ، $assign$ ، $ifTrue$ ، $ifFalse$) هرگز نمی‌تواند برابر $Stmt.while b body$ باشد. راه‌حل استاندارد (که در اثبات‌های $BigStep$ در Lean و Coq بسیار رایج است) این است که به‌جای استقرا مستقیم روی $hRun$ ، یک گزاره‌ی کمکی aux تعریف می‌کنیم که روی یک دستور عمومی S بیان شده، به‌همراه یک فرض تساوی جداگانه $S = Stmt.while b body$. حالا می‌توان روی اشتقاق $h : BigStep S s0 s1$ به‌طور کاملاً عمومی استقرا زد (چون S دیگر ثابت نیست)، و در هر شاخه، فرض تساوی را با $cases hEq$ باز کرد:

- در چهار شاخه‌ی seq ، $ifTrue$ و $ifFalse$ ، فرض hEq چیزی از این جنس است: $Stmt.assign x e = Stmt.while b body$. چون این دو طرف تساوی از دو سازنده‌ی متفاوت یک نوع استقرایی ساخته شده‌اند، چنین تساوی‌ای اساساً غیرممکن است. تاکتیک $cases hEq$ این تناقض را تشخیص می‌دهد و بلافاصله آن شاخه را می‌بندد — به همین دلیل بعد از $cases hEq$ در این چهار مورد هیچ خط دیگری لازم نیست؛ هدف با صفر اثبات باقی‌مانده بسته می‌شود.
- در شاخه‌ی $whileFalse$ ، hEq از نوع $Stmt.while b body' = Stmt.while b body$ است. اینجا $cases hEq$ تساوی $b' = b$ و $body' = body$ را استخراج و در کل هدف جایگزین می‌کند. طبق تعریف سازنده‌ی $whileFalse$ ، حالت پایانی همان حالت شروع است ($s1 = s0$)، پس هدف باقی‌مانده دقیقاً $P s0$ می‌شود که همان $hInv$ است.

- در شاخه‌ی $whileTrue$ ، بعد از $cases hEq$ همان جایگزینی رخ می‌دهد. حالا از $bodyPreservesInvariant$ (با سه آرگومان $hCond$ ، $hBody$ و $hInv$) نتیجه می‌گیریم که ناوردا بعد از یک بار اجرای بدنه هم برقرار است: $hInvAfterBody : P sMid$. سپس فرض استقرای مربوط به «ادامه‌ی حلقه» یعنی $ihRest$ را با دو آرگومان rfl (اثبات بدیهی تساوی $Stmt.while b body = Stmt.while b body$) و $hInvAfterBody$ صدا می‌زنیم تا $P sFinal$ نتیجه شود.

این الگو – عمومی کردن یک گزاره با افزودن یک فرض تساوی صریح، پیش از استقرا – در ادبیات اثبات‌یاریها به «ترفند remember» معروف است (در Coq معادل تاکتیک `remember ... as ...` eqn:...)، و هر جا بخواهیم از استقرا روی اشتقاق یک رابطه‌ی استقرایی که یکی از آرگومان‌هایش به یک عبارت مرکب و ثابت (نه یک متغیر ساده) گره خورده استفاده کنیم، ضروری است.

۳.۱۵.۵ اتصال نهایی

در پایان، قضیه‌ی اصلی `loopInvariant` تنها با یک خط `hP rfl hRun` `exact aux hRun` بسته می‌شود: `hRun` همان اشتقاق اجرای حلقه است، `rfl` گواهی بدیهی بر تساوی `Stmt.While b body = Stmt.While b` `body`، و `hP` فرض برقراری ناوردا در ابتدای حلقه.

۴.۱۵.۵ یک نکته‌ی فنی درباره‌ی کامپایل

دو دستور `BigStep` `#check` و `loopInvariant` `#check` – که صرفاً برای تأیید صحت `type` تعاریف و بدون اثر روی خود اثبات هستند – باید بیرون از بلوک `by` قضیه (یعنی بعد از خط `exact aux hRun` `hP rfl` و با تورفتگی سطح فایل) نوشته شوند، نه داخل بدنه‌ی تاکتیکی. در کد نهایی تحویلی، این دو خط به همین صورت اصلاح و در انتهای فایل، خارج از قضیه، قرار داده شده‌اند.

۱۶.۵ مثال گام‌به‌گام (شفاف‌سازی رفتار استقرا)

برای ملموس‌تر شدن مکانیزم بالا، اجرای یک حلقه‌ی فرضی با دقیقاً دو بار تکرار بدنه را در نظر بگیرید. اشتقاق `hRun` به این شکل درخت می‌شود:

$$\frac{\langle S, \sigma_0 \rangle \Downarrow \sigma_1 \quad \frac{\langle S, \sigma_1 \rangle \Downarrow \sigma_2 \quad \frac{\text{(شرط نادرست)}}{\langle ta(b)\{S\}, \sigma_2 \rangle \Downarrow \sigma_2} \text{(whileFalse)}}{\langle ta(b)\{S\}, \sigma_1 \rangle \Downarrow \sigma_2} \text{(whileTrue)}}{\langle ta(b)\{S\}, \sigma_0 \rangle \Downarrow \sigma_2} \text{(whileTrue)}$$

استقرا روی این درخت دقیقاً از پایین (بیرونی‌ترین `whileTrue`) شروع می‌شود: ابتدا با `bodyPreservesInvariant` از $P(\sigma_0)$ به $P(\sigma_1)$ می‌رسیم، سپس `ihRest` روی زیراشتقاق داخلی فراخوانی می‌شود که خودش دوباره یک `whileTrue` است – پس دوباره با `bodyPreservesInvariant` از $P(\sigma_1)$ به $P(\sigma_2)$ می‌رسیم، و در نهایت با `whileFalse` در عمیق‌ترین لایه، $P(\sigma_2)$ بدون تغییر به عنوان نتیجه‌ی نهایی برگردانده می‌شود. یعنی اثبات به تعداد تکرارهای واقعی حلقه، «باز» می‌شود – دقیقاً همان چیزی که استقرای ساختاری روی `BigStep` تضمین می‌کند، بدون آنکه لازم باشد تعداد تکرارها را از پیش بدانیم یا در قضیه ذکر کنیم.

۱۷.۵ جمع‌بندی فاز سوم

در این فاز، با تعریف صوری `Big-Step` برای زیرمجموعه‌ی `Hash` و اثبات قضیه‌ی پایستگی ناورداي حلقه در `Lean 4`، از سطح بررسی موردی فاز دوم (که فقط درباره‌ی یک برنامه‌ی خاص با `SPIN` صحبت می‌کرد) به یک تضمین ریاضی و کلی رسیدیم که برای هر برنامه‌ی `Hash` حاوی یک حلقه‌ی `ta` با بدنه‌ی ناوردا-حفظ‌کننده صادق است. این قضیه نمونه‌ی کوچکی از همان رویکردی است که، طبق مقدمه‌ی صورت پروژ، شرکت‌هایی مانند `Amazon` برای اثبات صحت زیرساخت‌های نرم‌افزاری‌شان به کار می‌برند: به جای تکیه بر تست‌های موردی، درستی را یک‌بار و برای همیشه، به صورت ریاضی، ثابت می‌کنند.